



SCHOOL OF COMPUTING AND INFORMATICS
DEPARTMENT OF INFORMATION TECHNOLOGY

PROJECT TITLE

**DESIGN AND EVALUATION OF A MACHINE LEARNING PIPELINE FOR
HEALTH INSURANCE CLAIMS VERIFICATION IN KENYA'S SOCIAL HEALTH
AUTHORITY (SHA)**

NAME: SICHEMO, Elvis Paul

REG. No: BIT/2019/42945

SUPERVISOR: CHOGLI, Bernard

Date: 28th July 2026

This project proposal is submitted in partial fulfilment of requirement for the
Mount Kenya University award of

BACHELOR OF SCIENCE IN INFORMATION TECHNOLOGY

DECLARATION

I hereby declare that this project proposal is based on my original work except for citations and quotations which have been duly acknowledged. I also declare that it has not been previously or concurrently submitted for any other degree or award at Mount Kenya University or any other institution.

Signature:

Name: Elvis Paul Sichemo

Admission Number: BIT/2019/42945

Date:

Supervisor declaration

I, the undersigned, do hereby certify that this is a true proposal for the project undertaken by the above-named student under my supervision and has been submitted to Mount Kenya University with my approval.

Name:

Signature: Date:

Contents	
PROJECT TITLE	0
DECLARATION	i
ABSTRACT	vii
DEDICATION	viii
ACKNOWLEDGEMENT	ix
LIST OF ABBREVIATIONS	x
CHAPTER ONE: INTRODUCTION	1
1.1 Background	1
1.2 Problem Statement	2
1.3 Proposed Solution	2
1.4 Project Objectives	4
1.4.1 Main Objective	4
1.4.2 Specific Objectives	4
1.5 Research Questions	4
1.6 Project Scope	5
1.6.1 In Scope	5
1.6.2 Out of Scope	5
1.7 Project Justification	6
1.8 Limitations and Assumptions	7
1.8 Project Risks and Mitigation Strategies	7
1.9 Project Budget	8
1.10 Project Schedule	9
1.10.1 Gantt Chart reference to appendix & budget	9
CHAPTER TWO: LITERATURE REVIEW	10
2.1 Introduction	10
2.3 Electronic Claims Processing Systems: A Critical Review	10
2.4 Machine Learning in Health Insurance Anomaly Detection	11
2.4.1 Gradient Boosting and XGBoost	11
2.4.2 Justification for a Single-Model Design	12
2.5 Explainable AI and SHAP	12
2.6 Automated Provider Monitoring and Sanctions	13
2.7 Ethical Considerations in AI-Driven Healthcare Systems	13
2.8 Critical Comparison of Existing Systems	14
2.9 Research Gap	15
CHAPTER THREE: METHODOLOGY	16
3.1 Introduction	16
3.2 Research Design	16
3.3 Study Area and Target Population	16
3.4 Data Collection Methods	17
3.4.1 Primary Data	17
3.4.2 Secondary Data	17

3.4.3 Ethical Data Collection Procedures.....	18
3.5 System Architecture	18
3.6 Machine Learning Pipeline	18
3.6.1 Data Pre-processing.....	18
3.6.2 Feature Engineering	19
3.6.3 Model Implementation and Training.....	21
3.6.4 SHAP Explainability Integration	21
3.6.5 Automated Provider Suspension Logic	21
3.6.6 Model Evaluation Strategy	21
3.7 Prototype Development	22
3.8 System Validation and Testing	22
3.9 Ethical Considerations	23
CHAPTER FOUR: SYSTEM ANALYSIS AND REQUIREMENTS MODELLING	24
4.1 Introduction.....	24
4.2 Analysis of the Current System.....	24
4.2.1 Stakeholder Findings.....	24
4.2.2 Identified Gaps abbreviations	25
4.3 Feasibility Study.....	26
4.3.1 Technical Feasibility	26
4.3.2 Economic Feasibility	26
4.3.3 Operational Feasibility	27
4.3.4 Schedule Feasibility	27
4.4 Description of the Current Claims Verification System.....	27
4.4.1 Current System Flowchart.	28
4.5 Data Flow Diagrams: Current System.....	29
4.5.1 Context Diagram (Level 0 DFD)	29
4.5.2 Level 1 DFD (Process Decomposition).....	29
4.6 Verification-Specific Limitations of the Current System.....	30
4.7 Proposed System Modelling	32
4.7.1 UML Class Diagram - Proposed XGBoost Verification Pipeline	32
4.7.2 UML Sequence Diagram - Claim Submission through XGBoost Verification to Officer Review	33
4.8 Requirements Definition - Current System	34
4.9 Requirements Definition and Specification - Proposed XGBoost Verification System.....	35
4.9.1 Functional Requirements.....	35
4.9.2 Non-Functional Requirements	36
4.10 Chapter Summary.....	36
CHAPTER FIVE: SYSTEM DESIGN	38
5.1 Introduction.....	38
5.2 System Architecture	38
5.2.1 Architectural Overview	38
5.3 Database Design.....	39
5.3.1 Entity Relationship Diagram.....	39

5.4 XGBoost Pipeline Design	41
5.4.1 Pipeline Stages	41
5.4.2 Key Features	41
5.4.3 Automated Provider Suspension Design	42
5.5 User Interface Design	43
5.5.1 Claim Submission sketch (Healthcare Provider)	43
5.5.2 Officer Review and SHAP Decision Sketch.....	43
5.6 Security Design	44
5.7 Report Generation Design	44
5.7.1 Report Types	45
5.8 Requirements Traceability	45
5.9 Chapter Summary.....	46
CHAPTER SIX: CODING AND DEVELOPMENT	47
6.1 Introduction	47
6.2 Development Environment and Tools	47
6.3 Coding Standards and Conventions	48
6.4 Project Structure	48
6.5 Database Implementation	49
6.6 Backend Implementation	49
6.6.1 Authentication and Role-Based Access Control.....	49
6.6.2 Claims Submission and Verification Endpoint.....	49
6.6.3 Officer Review Module	49
6.6.4 Administration Module.....	50
6.7 Machine Learning Pipeline Implementation.....	50
6.7.1 Feature Engineering	50
6.7.2 SHAP Explanation	50
6.7.3 Automated Provider Suspension Module	50
6.8 Frontend Implementation.....	51
6.9 Report Generation Implementation	51
6.10 Version Control and Configuration Management	51
6.11 Chapter Summary.....	52
CHAPTER SEVEN: TESTING THE MODELS	53
7.1 Introduction	53
7.2 Testing Strategy.....	53
7.3 Test Environment and Test Data.....	53
7.4 Unit Testing.....	54
7.5 Integration Testing	55
7.6 System Testing	56
7.6.1 Functional Testing.....	56
7.6.2 Performance Testing.....	56
7.6.3 Security Testing	56
7.7 Model Evaluation	56

7.7.1 Evaluation Results	56
7.7.2 Threshold Analysis.....	57
7.7.3 Explanation Consistency	57
7.8 User Acceptance Testing	57
7.9 Requirements Validation Matrix.....	58
7.10 Chapter Summary.....	59
CHAPTER EIGHT: CONCLUSION AND RECOMMENDATIONS.....	60
8.1 Introduction.....	60
8.2 Summary of the Project	60
8.3 Achievement of Objectives.....	60
8.4 Contributions of the Study.....	61
8.5 Limitations of the Study	62
8.6 Recommendations	62
8.7 Future Work	63
8.8 Conclusion	63
REFERENCES	65
Appendix A: Stakeholder Questionnaire	67
Section A: Respondent Information.....	67
Section B: Current Verification System Challenges.....	67
Section C: ML-Based Verification System Expectations	67
Appendix B: Interview Guide	69
Appendix C: System Requirements Specification	70
Functional Requirements.....	70
Non-Functional Requirements	71
Appendix D: Gantt Chart.....	72
Appendix E: User Manual	73
D.1 System Requirements and Installation.....	73
D.2 Signing In.....	73
D.3 Submitting a Claim (Provider)	73
D.4 Reviewing Flagged Claims (Officer)	73
D.5 Administration (Administrator).....	74
D.6 Reports	74
D.7 Troubleshooting.....	74
List of Tables	
Table 1: Abbreviations	xi
Table 2: Project Risks and Mitigation Strategies	8
Table 3: Project Budget	9
Table 4: Project Schedule	9
Table 5: Critical Comparison of Existing Claims Verification Systems.....	14
Table 6: Target Population and Sample Distribution.....	17
Table 7: Agile iterative cycles applied to feature engineering	20
Table 8: Engineered Feature Vector.....	21
Table 9: Prototype Technology Stack	22
Table 10: Identified Gaps in the Current System.	25
Table 11: Summary of Key Findings from Stakeholder Data Collection	26

Table 12: Verification Limitations of the Current System and XGBoost Pipeline Responses	31
Table 13: Current System Verification Requirements and Fulfilment Status.	34
Table 14: Functional Requirements - Proposed XGBoost Claims Verification System.....	35
Table 15: Non-Functional Requirements - Proposed XGBoost Claims Verification System	36
Table 16: Technology Stack	39
Table 17: XGBoost Verification Pipeline Stages.....	41
Table 18: Key XGBoost Feature Descriptions (selected from 17-feature vector).....	42
Table 19: Report Types and Content.....	45
Table 20: Development Tools and Versions.....	48
Table 21: Testing Levels and Objectives.....	53
Table 22: Unit Test Summary	55
Table 23: Integration Test Cases	55
Table 24: Model Performance on the Held-Out Test Set	57
Table 25: UAT Feedback Summary.....	58
Table 26: Requirements Validation Matrix	59
Table 27: Achievement of Specific Objectives.....	61
Table 28: Functional Requirements	71
Table 29: Non-Functional Requirements.....	71
Table 30: Gantt Chart	72

List of Figures

Figure 1: Current SHA Claims Verification Flowchart.....	28
Figure 2: Level 0 DFD. Current SHA Claims Verification System (Context Diagram).	29
Figure 3: Level 1 DFD: Current SHA Claims Verification System (Process Decomposition).....	30
Figure 4: UML Class diagram - Proposed XGBoost Claims Verification System.....	32
Figure 5: UML Sequence Diagram- Claim Submission through XGBoost Verification to Officer Decision.	33
Figure 6: Architecture Diagram	38
Figure 7: Entity Relationship Diagram.....	40
Figure 8: Claim Submission Screen sketch	43
Figure 9: Claim Review and Decision.....	43
Figure 10: Role-Based Access Control Matrix	44
Figure 11: Requirements Traceability Matrix.....	46

ABSTRACT

Kenya's Social Health Authority (SHA) manages a complex national health insurance framework that processes thousands of claims daily from healthcare providers across the country. The current claims management process relies heavily on rule-based digital checks and manual review, resulting in slow verification cycles, reimbursement delays that routinely exceed the statutory 14 to 90-day window, and significant exposure to erroneous and fraudulent claims that threaten the financial sustainability of the scheme.

This project presents the design, implementation, and evaluation of a machine learning pipeline for automated health insurance claims verification, built around a single, rigorously optimised XGBoost (Extreme Gradient Boosting) classification model. The pipeline performs end-to-end verification of each submitted claim: deterministic pre-checks confirm patient eligibility, provider accreditation, and coverage validity; a 17-feature engineered vector is then scored by the XGBoost model to estimate the probability that the claim is anomalous; and SHAP (SHapley Additive exPlanations) values are computed for every flagged claim so that human verification officers can see exactly which factors drove the model's decision. Claims scoring above a configurable threshold are routed to a human review queue in keeping with the human-in-the-loop principle, while an automated provider monitoring module suspends any healthcare provider that accumulates ten flagged claims within a rolling thirty-day window, with every suspension recorded in an append-only audit log.

The system is implemented as a four-tier architecture comprising a web-based presentation layer, a FastAPI application layer secured with JSON Web Token (JWT) authentication and role-based access control across four user roles (provider, verification officer, administrator, and finance officer), a Python machine learning pipeline layer, and a PostgreSQL data layer. The methodology follows a Design and Development Research approach, combining primary data collection from SHA stakeholders with iterative implementation and testing of the functional prototype. Model performance is evaluated using accuracy, precision, recall, F1-score, and AUC-ROC on a held-out test set, with SMOTE applied to address class imbalance during training.

The outcome is a validated, deployable machine learning pipeline that accelerates claims processing, reduces exposure to anomalous claims, provides transparent and explainable model decisions, and supports evidence-based decision-making within Kenya's universal health coverage agenda.

DEDICATION

This project is dedicated to my father for his unwavering support, encouragement, and patience throughout my academic journey, and to all healthcare workers and administrators striving to make universal health coverage a reality for every Kenyan.

ACKNOWLEDGEMENT

I sincerely thank the Almighty God for the gift of life, health, and strength throughout the course of this project. I extend my deepest gratitude to my supervisor, Mr. CHOGI, Bernard, for his invaluable guidance, constructive criticism, and steady encouragement at every stage of this work. I also thank the lecturers and staff of the School of Computing and Informatics, Mount Kenya University, for the knowledge and skills imparted throughout my studies. Special appreciation goes to the SHA officers, healthcare providers, system administrators and Turquoise Solutions Limited; who generously gave their time and insight during data collection. Finally, I thank my family for their moral support and encouragement.

LIST OF ABBREVIATIONS

Abbreviation	Full Form
API	Application Programming Interface
AUC-ROC	Area Under the Receiver Operating Characteristic Curve
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
CSV	Comma-Separated Values
DDR	Design and Development Research
DFD	Data Flow Diagram
ERD	Entity Relationship Diagram
FastAPI	Fast Application Programming Interface (Python web framework)
GDPR	General Data Protection Regulation
ICD-10	International Classification of Diseases, 10th Revision
JSON	JavaScript Object Notation
JWT	JSON Web Token
KMPDC	Kenya Medical Practitioners and Dentists Council
MKU	Mount Kenya University
ML	Machine Learning
NCK	Nursing Council of Kenya
NHIF	National Health Insurance Fund
ORM	Object-Relational Mapping
PDF	Portable Document Format
PPB	Pharmacy and Poisons Board
RBAC	Role-Based Access Control
REST	Representational State Transfer
SHA	Social Health Authority
SHAP	SHapley Additive exPlanations
SHIF	Social Health Insurance Fund
SMOTE	Synthetic Minority Over-sampling Technique

SUS	System Usability Scale
TLS	Transport Layer Security
UAT	User Acceptance Testing
UHC	Universal Health Coverage
UML	Unified Modelling Language
XGBoost	Extreme Gradient Boosting
CVR	Claims Verification Requirement

Table 1: Abbreviations

CHAPTER ONE: INTRODUCTION

1.1 Background

Kenya's healthcare financing landscape underwent a significant transformation in 2023 with the establishment of the Social Health Authority (SHA) to replace the National Health Insurance Fund (NHIF) under the Social Health Insurance Act (No. 16 of 2023). SHA administers the Social Health Insurance Fund (SHIF), the Primary Healthcare Fund, and the Emergency, Chronic and Critical Illness Fund, and is mandated to deliver Universal Health Coverage (UHC) to all Kenyan residents. The authority processes thousands of claims daily submitted by empanelled healthcare providers ranging from Level 2 dispensaries to Level 6 national referral hospitals.

Despite this institutional reform, the operational challenges that undermined NHIF including fraudulent claims, slow verification processes, and delayed provider reimbursements, have persisted and in some cases, intensified during the transition period. Healthcare providers continue to report reimbursement delays that strain their operational cash flow, while SHA continues to face exposure to inflated, duplicate, and otherwise anomalous claims that erode the financial base of the scheme.

A key operational gap is the absence of an intelligent, automated claims verification mechanism. Current systems rely on rule-based digital processing and manual review, which are slow, inconsistent, and unable to detect the subtle, multivariate patterns that characterise problematic claims. Verification officers are overwhelmed by claim volumes, and no systematic machine learning capability exists within the claims pipeline to prioritise their attention towards the claims most likely to be irregular.

This project therefore proposes a focused, technically grounded response: ***the design, implementation, and evaluation of a machine learning pipeline for health insurance claims verification, tailored to the data structures, regulatory context, and operational realities of Kenya's Social Health Authority.*** The pipeline is built around a single, rigorously optimised XGBoost classification model augmented with SHAP explainability, deterministic eligibility pre-checks, human-in-the-loop review of flagged claims, and an automated provider monitoring module that suspends providers exhibiting sustained patterns of flagged submissions.

1.2 Problem Statement

Kenya's Social Health Authority faces a critical operational challenge in managing the integrity and efficiency of health insurance claims. The existing claims management framework suffers from the following interrelated problems:

1. **Inadequate anomaly detection:** The current system relies on basic rule-based checks that are insufficient to detect sophisticated anomalous patterns such as ghost patients, duplicate claims across facilities, inflated billing relative to standard tariffs, and diagnosis-procedure mismatches.
2. **Manual and slow claims verification:** Claims verification is largely manual, leading to processing delays that exceed the statutory 14 to 90-day reimbursement window and causing healthcare providers to experience severe cash-flow strain.
3. **Absence of machine learning integration:** No systematic application of machine learning exists within SHA's claims pipeline. Without a trained predictive model, the system cannot identify anomalies, learn from historical decisions, or prioritise officer attention towards the highest-risk claims.
4. **Lack of decision explainability:** Even where automated scoring exists in comparable systems, officers are reluctant to trust opaque model outputs. Without an explanation of why a claim was flagged, human reviewers cannot efficiently validate or overturn algorithmic decisions, undermining adoption.
5. **No systematic provider-level monitoring:** There is no automated mechanism to detect and act upon providers who exhibit sustained patterns of anomalous submissions; enforcement is reactive, slow, and inconsistent.
6. **Patient data privacy risks:** The handling of sensitive health data lacks a formally documented ethical framework, creating vulnerabilities in data governance and compliance with the Data Protection Act (2019).

These problems collectively compromise the financial sustainability of SHA and undermine public trust in Kenya's universal healthcare system.

1.3 Proposed Solution

This project proposes the development of a unified machine learning claims verification pipeline organised around five tightly integrated capabilities:

1. **Deterministic pre-verification:** Every submitted claim first passes through fast, rule-based gates that confirm patient eligibility and coverage validity against the SHA member register, verify provider accreditation and empanelment status, and confirm that the diagnosis-procedure combination exists in the SHA tariff schedule. Claims failing these checks are rejected immediately with a documented reason, without consuming model capacity.
2. **XGBoost anomaly scoring:** Claims passing pre-verification are transformed into a 17-feature engineered vector capturing billing behaviour (such as the ratio of claimed amount to SHA tariff), provider history (such as 30-day claim frequency and peer-relative amount z-scores), temporal patterns (such as submission delay), and clinical consistency (such as diagnosis-procedure match). A single XGBoost classification model, selected for its state-of-the-art performance on imbalanced tabular data, scores each claim with a probability that it is anomalous. Claims above a configurable threshold are flagged.
3. **SHAP explainability:** For every flagged claim, SHAP (SHapley Additive exPlanations) values are computed and the top five contributing features are stored and displayed to the verification officer in plain language. This converts the model from an opaque black box into a transparent decision-support tool, directly addressing the trust barrier identified in the problem statement.
4. **Human-in-the-loop review:** Flagged claims are routed to a review queue where SHA verification officers examine the claim, its pre-check results, and its SHAP explanation before approving, rejecting, or escalating it. Officer comments are mandatory, and every action is written to an append-only audit log. The model never rejects a claim autonomously.
5. **Automated provider suspension:** A provider monitoring module tracks flagged claims per provider over a rolling thirty-day window. Any provider accumulating ten flagged claims within the window is automatically suspended from submitting further claims, with the suspension recorded in the audit log and reversible only by an administrator who must document a reason. This shifts provider enforcement from reactive to proactive.

The pipeline includes structured data pre-processing, feature engineering, model training with hyperparameter tuning, stratified cross-validation, SMOTE-based class imbalance treatment, and rigorous performance evaluation. The system is delivered as a functional prototype

comprising a PostgreSQL database, a FastAPI backend secured with JWT authentication and role-based access control, and a web-based user interface serving four user roles.

1.4 Project Objectives

1.4.1 Main Objective

To design, implement, and evaluate a machine learning pipeline for automated health insurance claims verification within Kenya's Social Health Authority.

1.4.2 Specific Objectives

1. To analyse the current claims verification processes within SHA and identify key system gaps and requirements.
2. To design a data pre-processing and feature engineering pipeline producing a 17-feature vector suitable for structured health insurance claims data.
3. To implement and train an XGBoost classification model for claims anomaly scoring using historical and synthetic SHA claims data, with SMOTE applied to address class imbalance.
4. To integrate SHAP explainability so that every flagged claim is accompanied by a ranked, human-readable explanation of the features that drove its score.
5. To develop a functional prototype demonstrating end-to-end claims verification including provider claim submission, officer review with mandatory audit logging, administrative CRUD operations, reporting, and automated provider suspension on a FastAPI and PostgreSQL stack secured with JWT authentication and role-based access control.
6. To evaluate model performance using accuracy, precision, recall, F1-score, and AUC-ROC on a held-out test set.
7. To document and address ethical considerations including patient data privacy, informed consent, algorithmic bias, and data governance compliance.

1.5 Research Questions

1. What are the primary inefficiencies and vulnerabilities in SHA's current claims verification and management processes?
2. Which engineered features of a health insurance claim are most predictive of anomalous submissions in the SHA context?

3. What level of classification performance (precision, recall, F1-score, AUC-ROC) can a tuned XGBoost model achieve on SHA-structured claims data under realistic class imbalance?
4. How can SHAP explanations be presented so that verification officers can efficiently validate or overturn model decisions?
5. What flagged-claims threshold over what time window constitutes a defensible trigger for automatic provider suspension?
6. What ethical safeguards must be embedded in an AI-driven claims verification system to protect patient data privacy and ensure algorithmic fairness?

1.6 Project Scope

This project is deliberately scoped to a single, deeply implemented verification pipeline to ensure depth of implementation and rigour of evaluation within the available project timeframe.

1.6.1 In Scope

- Claims verification automation: patient eligibility, coverage validation, diagnosis-procedure tariff matching, and provider accreditation checks.
- Machine learning anomaly scoring: training, tuning, and evaluation of a single XGBoost classification model over a 17-feature engineered vector.
- SHAP explainability: computation and plain-language display of the top five features driving each flagged claim's score.
- Data pipeline: pre-processing, feature engineering, handling of class imbalance (SMOTE), and stratified cross-validation.
- Automated provider suspension: rolling 30-day flag counting with a threshold of ten flagged claims, append-only audit logging, and administrator-controlled reinstatement.
- Prototype system with role-based interfaces for providers, verification officers, administrators, and finance officers, including claim submission, review queue, administrative CRUD, and PDF/CSV reporting.
- Ethical framework: data privacy measures, bias assessment, and data governance documentation.

1.6.2 Out of Scope

- Full hospital Electronic Medical Record (EMR) system development.

- Comparative evaluation of multiple machine learning algorithms; the project commits to a single, deeply optimised XGBoost model.
- Predictive healthcare cost forecasting and broader financial analytics dashboards.
- Integration with live SHA production systems or national patient registries.
- Direct clinical decision support or patient-facing mobile interfaces.
- International insurance system integration.

1.7 Project Justification

The justification for this project is grounded in the operational reality of SHA and the demonstrated effectiveness of machine learning in comparable health insurance environments.

First, the financial stakes are significant. Fraudulent and erroneous health insurance claims represent a global problem estimated to account for 3–10% of total healthcare expenditure. In the Kenyan context, where SHA's mandate covers the entire resident population, even a small proportion of anomalous claims translates into substantial losses that directly undermine the universal health coverage agenda.

Second, gradient-boosted tree models have demonstrably outperformed rule-based systems and other classifiers in insurance anomaly detection. Benchmarking studies consistently show XGBoost achieving superior AUC-ROC on imbalanced tabular insurance data, while its regularisation controls overfitting and its computational efficiency supports the sub-3-second scoring latency the system requires. Committing to a single model rather than a comparative study permits deeper hyperparameter optimisation, full explainability integration, and complete end-to-end system delivery within the project timeframe.

Third, explainability is the adoption bottleneck that most deployed systems neglect. By pairing every flagged claim with SHAP-derived reasons, the system directly addresses officer trust; a requirement that emerged strongly from stakeholder engagement and that the literature identifies as decisive for real-world uptake of AI decision support.

Fourth, the academic contribution is clear. There is limited published research applying explainable machine learning verification to Kenya's health insurance claims context, and none documenting an automated provider suspension mechanism with auditable human oversight. This project contributes a documented, evaluated reference implementation.

Finally, the school's supervision guidelines emphasise functional system development with CRUD operations, reporting capabilities, and evidence of progressive development; all of

which this project satisfies through its administrative interfaces, PDF and CSV reporting module, versioned Git repository, and staged delivery plan.

1.8 Limitations and Assumptions

The following limitations and working assumptions bound the study:

- **Data access:** Live SHA production data was not available for research use; the model is trained and evaluated on synthetic data generated to reflect SHA's schema together with public insurance datasets. Reported performance is therefore indicative, and retraining on operational data is a precondition for deployment.
- **Deployment context:** The prototype is evaluated in a development environment rather than under production load; performance targets are verified at prototype scale.
- **Label fidelity:** Anomaly labels in the synthetic dataset are injected according to documented patterns (inflated billing, duplicate submissions, tariff mismatches); real-world anomaly distributions may differ.
- **Reference data:** The study assumes SHA's tariff schedule and member register are authoritative and reasonably current, as verification decisions are made against them.
- **Sampling:** Stakeholder sampling is purposive and small (18 respondents); findings inform requirements rather than support statistical generalisation.

1.8 Project Risks and Mitigation Strategies

Risk	Description	Mitigation
Data availability and quality	SHA operational data may be unavailable for research use due to privacy restrictions.	Synthetic data generation reflecting SHA's schema, plus publicly available insurance datasets (e.g., CMS Medicare) for initial development and benchmarking.
Class imbalance in anomaly data	Anomalous claims typically represent a small minority of all claims, causing models to under-detect them.	SMOTE applied to the training set only, plus threshold tuning on the validation set and evaluation with imbalance-robust metrics (F1, AUC-ROC).

Model overfitting	The model may perform well on training data but generalise poorly to unseen claims.	Stratified 5-fold cross-validation, a held-out test set untouched until final evaluation, and XGBoost's built-in L1/L2 regularisation.
Officer distrust of model decisions	Verification officers may ignore or override flags they cannot understand, negating system value.	SHAP explanations rendered in plain language for every flagged claim; mandatory officer comments create a feedback record for future model refinement.
Erroneous automatic suspension	A provider could be suspended due to model false positives rather than genuine misconduct.	Conservative threshold (10 flags/30 days), every suspension logged to an append-only audit trail, and administrator-controlled reinstatement with mandatory documented reason.
Ethical and privacy risks	Use of patient health data raises privacy concerns and potential for algorithmic bias.	Data anonymisation, role-based access control, TLS encryption, and bias auditing across demographic and geographic variables.

Table 2: Project Risks and Mitigation Strategies

1.9 Project Budget

Budget Item	Description	Amount (KES)
Data Collection	Printing questionnaires, travel to SHA offices and healthcare facilities for interviews and observation	8,000
Software and ML Tools	Python libraries (XGBoost, scikit-learn, SHAP, pandas, matplotlib), IDE licences, cloud notebook access	5,000
Cloud Storage and Hosting	Dataset storage, model hosting, prototype web application deployment	5,000
System Development	Backend API development, database design, XGBoost pipeline coding, SHAP integration, dashboard frontend	10,000
System Testing	Unit testing, integration testing, model evaluation, user acceptance testing with stakeholders	5,000
Documentation and Printing	Final report printing, proposal printing, ethics forms, data agreements	7,000
Presentation Materials	Slides, poster (if required), defence preparation materials	5,000

Miscellaneous	Unforeseen expenses	5,000
TOTAL ESTIMATED BUDGET		50,000 KES

Table 3: Project Budget

1.10 Project Schedule

The project will be implemented over a 16-week period using Agile methodology with two-week sprints, aligned to the minimum six supervision sessions required by the School of Computing and Informatics.

Timeline	Activities
Weeks 1–2	Proposal finalisation, supervisor onboarding, ethics review submission, literature review
Weeks 3–4	Stakeholder data collection (questionnaires, interviews), current system analysis, requirements specification
Week 5	Database schema design, system architecture design, API contract definition
Weeks 6–7	Data pre-processing pipeline and 17-feature engineering module implementation; dataset preparation with SMOTE
Week 8	XGBoost model implementation, initial training, hyperparameter grid search, stratified cross-validation
Week 9	Model evaluation on held-out test set: accuracy, precision, recall, F1-score, AUC-ROC; SHAP explainer integration
Weeks 10–11	Prototype backend development: FastAPI application, JWT authentication, RBAC, claims submission and verification endpoints
Weeks 11–12	Automated provider suspension module; officer review queue; administrative CRUD; PDF/CSV reporting module
Week 13	Frontend integration against live API; system integration testing; user acceptance testing with stakeholders
Week 14	Threshold calibration, system refinement based on test results, security review
Week 15	Report writing: implementation, results, and conclusion chapters
Week 16	Final report submission, presentation preparation, defence rehearsal

Table 4: Project Schedule

1.10.1 Gantt Chart reference to appendix & budget

Refer to Appendix D figure 29.

CHAPTER TWO: LITERATURE REVIEW

2.1 Introduction

This chapter critically examines existing literature on health insurance claims management, machine learning-based anomaly detection, explainable artificial intelligence, and automated claims verification systems. Unlike a purely descriptive survey, the review evaluates the strengths and limitations of prior approaches in order to position the design decisions made in this project; in particular, the commitment to a single XGBoost model, the integration of SHAP explainability, and the introduction of automated provider-level monitoring within the existing body of knowledge.

2.2 Evolution of Health Insurance Management in Kenya

The National Health Insurance Fund (NHIF), established under Cap 255 of the Laws of Kenya, served as the country's primary public health insurer for decades before its replacement by the Social Health Authority. Over its lifetime, NHIF was repeatedly criticised for administrative inefficiency, weak claims controls, and susceptibility to fraudulent claims, with audit reports documenting losses through inflated invoices, claims for services never rendered, and collusion between facility staff and claims processors (Doetinchem, Carrin & Evans, 2010; Ministry of Health, 2020).

A critical gap identified in the literature is the absence of machine learning integration in Kenya's public health insurance systems. While NHIF implemented digital claim submission, its verification remained deterministic and static, unable to adapt to evolving anomaly patterns or to prioritise scarce reviewer attention. This project's pipeline is designed precisely for this institutional context: it augments, rather than replaces, the deterministic checks with a learned scoring layer and explicit human oversight.

2.3 Electronic Claims Processing Systems: A Critical Review

Electronic claims processing systems (ECPS) have been widely adopted across health insurance institutions globally. These systems enable digital claim submission, automated eligibility checks, and electronic payment processing, and have been shown to reduce administrative costs and turnaround times relative to paper-based processes (Raghupathi & Raghupathi, 2014).

Witten et al. (2016) argue that rule-based systems are fundamentally limited in high-dimensional, complex datasets such as health insurance claims, where problematic patterns are subtle, multivariate, and evolving. This motivates the layered design adopted in this project: deterministic rules retain their proper role as fast, transparent gates for unambiguous failures (ineligible patient, unaccredited provider, missing tariff), while a learned model handles the multivariate judgement that rules cannot express.

2.4 Machine Learning in Health Insurance Anomaly Detection

The application of machine learning to health insurance anomaly and fraud detection has been an active research area since the early 2010s. Ngai et al. (2011) provide a foundational classification framework for data mining techniques in financial fraud detection, identifying classification, clustering, and outlier detection as the dominant paradigms. Within classification, studies have evaluated decision trees, logistic regression, support vector machines, neural networks, and ensemble methods. Joudaki et al. (2015) survey data mining approaches specifically for health insurance fraud and abuse detection, concluding that supervised ensemble methods trained on labelled historical claims consistently outperform unsupervised alternatives when labelled data is available, and highlighting feature engineering quality as the strongest determinant of practical detection performance.

2.4.1 Gradient Boosting and XGBoost

XGBoost (Extreme Gradient Boosting) has emerged as a state-of-the-art algorithm for structured, tabular data classification. Introduced by Chen and Guestrin (2016), XGBoost builds an ensemble of decision trees sequentially, with each tree correcting the residual errors of its predecessors, and incorporates L1 and L2 regularisation directly into its objective function to control overfitting. Its scalable, cache-aware implementation supports the low-latency inference required in transactional settings such as claims processing. Justified alignment.

Multiple benchmarking studies have demonstrated XGBoost's strong performance on insurance datasets with severe class imbalance. Bauder, Khoshgoftaar and Seliya (2017), surveying healthcare upcoding fraud detection on US Medicare data, report ensemble tree methods achieving AUC-ROC values above 0.90, substantially outperforming logistic regression baselines. XGBoost's native handling of missing values, its support for class weighting, and its compatibility with SMOTE-resampled training data make it particularly

suited to claims data, where anomalous cases are rare and record completeness varies across facilities.

2.4.2 Justification for a Single-Model Design

Earlier iterations of this project proposal contemplated a comparative evaluation of Random Forest and XGBoost. The final design commits to a single, deeply optimised XGBoost model, for the following reasons:

1. The evidence base already strongly favours gradient boosting on imbalanced tabular insurance data; repeating a well-established comparison would consume project time without generating novel insight for the SHA context.
2. A single-model commitment permits substantially deeper work on what actually determines deployed performance: feature engineering (17 domain-informed features), hyperparameter optimisation via grid search with stratified cross-validation, threshold calibration, and SHAP explainability integration.
3. Interpretability, the traditional argument for Random Forest is addressed more rigorously by SHAP, which provides theoretically grounded, per-prediction attributions for the XGBoost model, exceeding what Random Forest feature importances offer.
4. The freed capacity funds a genuinely novel system contribution: the automated provider suspension module with auditable human oversight, which no reviewed system implements.

2.5 Explainable AI and SHAP

A consistent finding across deployment studies is that detection accuracy alone does not determine adoption: human reviewers must be able to understand and contest model decisions. Lundberg and Lee (2017) introduced SHAP (SHapley Additive exPlanations), a unified framework for interpreting model predictions grounded in cooperative game theory. SHAP assigns each feature an additive contribution to a specific prediction, with guarantees of local accuracy and consistency that heuristic importance measures lack. The TreeSHAP variant computes exact Shapley values for tree ensembles such as XGBoost in polynomial time, making per-claim explanation computationally feasible at transaction latency.

In this project, SHAP is not an analytical afterthought but a functional system requirement: every flagged claim is stored with its full SHAP vector and a pre-computed ranking of the top

five contributing features, rendered in plain language on the officer review screen (for example, 'Claimed amount is 4.2 times the SHA tariff' rather than 'amount_ratio = +0.31'). This design operationalises the human-in-the-loop principle: the model recommends and explains; the human decides.

2.6 Automated Provider Monitoring and Sanctions

Provider-level enforcement in the reviewed public insurance systems is overwhelmingly reactive: investigations are triggered by tip-offs, periodic audits, or accumulated complaints, often months after anomalous billing begins. The literature on programme integrity (Bauder et al., 2017) recommends continuous provider-level risk profiling aggregating claim-level signals into rolling provider statistics but few documented systems close the loop from profiling to automated action.

This project implements that closed loop conservatively. Claim-level flags aggregate into a rolling 30-day count per provider; crossing a threshold of ten flags triggers automatic suspension of the provider's submission capability. Three safeguards address the obvious risk of erroneous sanction: the threshold is deliberately conservative and configurable; every suspension is written to an append-only audit log with the triggering claim recorded; and reinstatement is an administrator-only action requiring a documented reason, with the provider's risk tier remaining elevated after reinstatement. The mechanism converts the model's claim-level outputs into proportionate, auditable, reversible institutional action.

2.7 Ethical Considerations in AI-Driven Healthcare Systems

The integration of AI into health insurance management raises significant ethical concerns that the literature increasingly identifies as central to responsible deployment, not optional add-ons. Three themes dominate:

- **Patient data privacy:** Health insurance claims contain highly sensitive personal and medical information. Kenya's Data Protection Act (2019) imposes strict obligations on organisations processing personal health data, including lawful basis, data minimisation, and security safeguards. This project responds with application-layer encryption of member names, TLS for all transmissions, role-based access control, and the exclusive use of anonymised or synthetic data during development.
- **Algorithmic fairness and bias:** ML models trained on historical data may perpetuate existing biases. In the SHA context, this could manifest as systematic over-flagging of

claims from certain counties or facility tiers. The evaluation plan includes disparate impact analysis across geographic and facility-type variables, with findings documented in the final report.

- **Human oversight:** Following the principle of human-in-the-loop decision-making for high-stakes outcomes, the system flags suspicious claims for human review rather than automatically rejecting them; the sole automated sanction (provider suspension) is threshold-based, logged, and reversible by a human administrator with a documented reason.

2.8 Critical Comparison of Existing Systems

System	Strengths	Limitations
Kenya NHIF electronic system	Digital claim submission, basic eligibility checks	Rule-based only; no ML scoring; no explainability; limited analytics; persistent delays
US Medicare Fraud Prevention System	ML anomaly detection, predictive analytics at national scale	Resource-intensive; requires large annotated datasets unavailable in Kenya; limited published explainability
South Africa GEMS system	Automated verification, anomaly detection	Proprietary; not publicly documented; different regulatory context
Academic ML prototypes (Bauder et al., 2017)	Rigorous ML evaluation on Medicare data	Not deployed; US-specific data; no verification workflow, explainability layer, or provider sanctions
Proposed SHA ML pipeline (this project)	Single optimised XGBoost model; SHAP per-claim explainability; human-in-the-loop review with append-only audit; automated provider suspension with administrator-controlled reinstatement; full RBAC prototype	Prototype only (not production); trained on synthetic/public data pending SHA data access; single-country scope

Table 5: Critical Comparison of Existing Claims Verification Systems

2.9 Research Gap

The literature review reveals a clear and significant research gap: while ML-based anomaly detection has been extensively studied and deployed in high-income country health insurance systems, there is minimal published work applying explainable, workflow-integrated machine learning verification to Kenya's public health insurance context and none documenting an automated, auditable provider suspension mechanism.

Furthermore, existing academic studies tend to stop at model evaluation, leaving unaddressed the system-level questions that determine real-world value: how explanations are surfaced to officers, how decisions are audited, and how claim-level signals translate into provider-level action. This project addresses both gaps by delivering and evaluating a complete, explainable verification pipeline from claim submission through XGBoost scoring, SHAP explanation, human review, and automated provider monitoring implemented on a realistic SHA data schema.

CHAPTER THREE: METHODOLOGY

3.1 Introduction

This chapter describes the research design, data collection strategy, system architecture, machine learning pipeline, and evaluation framework that guide the implementation of the proposed system. The methodology is structured to satisfy both the research rigour expected of an academic project and the engineering discipline required to deliver a functional, testable prototype.

3.2 Research Design

The study adopts a Design and Development Research (DDR) approach, which is appropriate when the primary research output is an engineered artefact developed to solve a real-world problem. The research combines qualitative methods; stakeholder interviews and document analysis to understand the claims verification context, with quantitative methods, ML model training, cross-validation, and performance metric evaluation.

The research proceeds through three sequential phases:

1. Analysis Phase: Investigating SHA's current claims verification processes, data structures, and stakeholder requirements through primary data collection and document review.
2. Design and Development Phase: Designing the system architecture, building the data preprocessing and feature engineering pipeline, implementing and training the XGBoost model, and developing the prototype application.
3. Evaluation Phase: Rigorous testing of XGBoost model performance, functional testing of the prototype, and stakeholder user acceptance testing.

3.3 Study Area and Target Population

The study focuses on the health insurance claims verification ecosystem managed by SHA in Kenya. The target population includes stakeholders directly involved in claims verification and management:

Respondent Category	Sample Size	Justification
SHA Claims Verification Officers	5	Primary insight into manual verification challenges, fraud patterns encountered, and officer information needs
Healthcare Providers (Hospitals/Clinics)	5	Represent the primary claim submitters; identify submission pain points and data quality issues
Healthcare Financial Officers	5	Perspective on reimbursement delays, financial impact of verification failures, and reporting needs
Health Information System Administrators	3	Technical insight into current data systems, API capabilities, and integration constraints
TOTAL	18	

Table 6: Target Population and Sample Distribution

Purposive sampling will be used to select respondents with direct, relevant experience in SHA claims verification ensuring technically informed data collection appropriate to the specialised domain.

3.4 Data Collection Methods

3.4.1 Primary Data

Structured questionnaires are administered to healthcare providers and SHA officers to collect quantitative data on current claim processing times, frequency of anomaly incidents, and perceived system weaknesses (Appendix D). Semi-structured interviews (Appendix E) provide deeper qualitative insight into verification workflows, trust requirements for AI-assisted decisions, and expectations for provider-level enforcement.

3.4.2 Secondary Data

Document review covers SHA operational guidelines, claims processing policy documents, published tariff schedules, and health financing reports. For ML model training, the study uses a combination of:

- anonymised synthetic claims data generated to reflect SHA's data schema, including realistic distributions of diagnosis codes, facility tiers, tariff amounts, and injected anomaly patterns; and
- publicly available health insurance datasets (e.g., the CMS Medicare Provider Utilisation dataset) for initial model development and benchmarking.

3.4.3 Ethical Data Collection Procedures

All primary data collection is preceded by informed consent. Respondent anonymity is protected in all data reporting. Research ethics clearance is sought from the MKU Research Ethics Committee before any primary data collection commences.

3.5 System Architecture

The system follows a modular, four-tier architecture designed to separate concerns and facilitate independent development, testing, and future scaling:

1. **Presentation layer:** A web-based interface serving four user roles; healthcare provider, SHA verification officer, administrator, and finance officer. It provides claim submission with live tariff lookup, the officer review queue with SHAP explanations, administrative CRUD screens with provider flag monitoring, and reporting with PDF and CSV export. The interface communicates with the application layer exclusively through authenticated REST calls.
2. **Application layer:** A FastAPI (Python) application exposing REST endpoints for authentication, claims, review, administration, reporting, and suspension management. Security is enforced through JSON Web Token (JWT) authentication with bcrypt password hashing and role-based access control (RBAC) applied per endpoint. Cross-Origin Resource Sharing (CORS) is restricted to approved origins.
3. **Machine learning pipeline layer:** A Python pipeline comprising the data pre-processing module, the 17-feature engineering module, the XGBoost inference engine, the SHAP explainer, and the automated provider suspension module. The pipeline is invoked synchronously on claim submission and loads its trained model artefact lazily so the API remains available before training completes.
4. **Data layer:** A PostgreSQL 15 relational database storing system users, SHA members, health providers, tariffs, claims, verification results (including full SHAP vectors), and an append-only audit log protected by revoked UPDATE/DELETE privileges. Generated columns compute the amount ratio and submission delay at insert time.

3.6 Machine Learning Pipeline

3.6.1 Data Pre-processing

Raw claims data undergoes the following pre-processing steps:

- **Missing value handling:** Median imputation for continuous variables; mode imputation for categorical variables; flag columns added for imputed values to preserve the missingness signal.
- **Categorical encoding:** One-hot encoding for nominal variables (facility type, diagnosis category); ordinal encoding for ordered variables (facility tier).
- **Feature scaling:** Log transformation of monetary amounts to compress heavy right tails; standard scaling for continuous features.
- **Class imbalance treatment:** SMOTE (Synthetic Minority Over-sampling Technique) applied to the training set only never to validation or test data to address the inherent class imbalance, where anomalous claims typically constitute under 5% of records.

3.6.2 Feature Engineering

Feature engineering for this project followed an Agile framework: an iterative, incremental approach in which work is delivered in short cycles, each informed by feedback from the cycle before it, rather than a single fixed specification completed upfront. This fits the nature of feature engineering directly, since a candidate feature's value is rarely obvious in advance and is usually only established once it has been tested against the model. It is also consistent with the Agile, sprint-based delivery already adopted for the project as a whole (Section 1.10) and with the Design and Development Research methodology described in Section 3.2, which treats the machine learning pipeline as an artefact refined through successive rounds of design and evaluation.

In practice, the feature set was built up over three short iterative cycles rather than fixed in a single pass:

Cycle	Focus	Outcome
Iteration 1	Domain-driven candidate generation	Candidate features drawn directly from the verification gaps identified in Chapter Four (for example, amount_ratio from the billing-compliance gap CVR-01, and patient_facility_count_7d from the ghost-patient pattern noted under provider anomaly detection).
Iteration 2	Importance-informed refinement	Preliminary XGBoost feature-importance scores from early training runs used to drop low-value candidates and

		adjust encodings (for example, collapsing rarely-varying categorical levels) before the next round of training.
Iteration 3	Explainability-informed refinement	SHAP value consistency checks confirmed that each retained feature contributed a coherent, domain-plausible explanation to flagged claims, closing the loop before the 17-feature vector was locked.

Table 7: Agile iterative cycles applied to feature engineering

This iterative process concluded once the 17-feature vector produced no further meaningful gain in preliminary validation performance. From that point, the feature set was frozen: no further changes were made once training moved against the held-out test partition, preserving the evaluation integrity described in Section 3.6.6. The resulting feature vector is presented in Table 7.

	Feature	Description
1	amount_ratio	Claimed amount divided by the SHA tariff for the diagnosis-procedure-tier combination
2	provider_claim_freq_30d	Number of claims submitted by the provider in the trailing 30 days
3	provider_amount_zscore	Z-score of the provider's average claim amount relative to peers in the same facility tier and county
4	diagnosis_procedure_match	Binary indicator that the diagnosis-procedure pair exists in the tariff schedule
5	submission_delay_days	Days between service date and claim submission
6	patient_facility_count_7d	Distinct facilities at which the patient generated claims in the trailing 7 days
7	provider_age_days	Days since the provider's empanelment date
8	facility_type_encoded	Encoded facility type (hospital, clinic, pharmacy, laboratory)
9	county_encoded	Encoded county of the provider
10	diagnosis_category_encoded	Encoded ICD-10 chapter of the diagnosis
11	claimed_amount_log	Natural log of the claimed amount
12	tariff_amount_log	Natural log of the applicable SHA tariff
13	provider_approval_rate	Historical share of the provider's claims approved

14	patient_claim_count_90d	Number of claims by the patient in the trailing 90 days
15	service_day_of_week	Day of week of the service date
16	coverage_package_encoded	Encoded SHA coverage package of the member
17	repeat_diagnosis_flag	Indicator that the patient received the same diagnosis within the trailing 30 days

Table 8: Engineered Feature Vector

3.6.3 Model Implementation and Training

A single XGBoost classifier is implemented using the xgboost Python library. Key hyperparameters tuned via grid search with stratified 5-fold cross-validation include: learning rate (eta: 0.01–0.3), maximum tree depth (max_depth: 3–10), number of estimators (n_estimators: 100–600), minimum child weight, subsample and column-sample ratios, L1 regularisation (alpha: 0–1), and L2 regularisation (lambda: 0–1). The positive-class weight (scale_pos_weight) is set from the observed class ratio to complement SMOTE. Experiments are tracked with MLflow, and the winning model artefact is versioned and persisted with joblib for lazy loading by the application layer.

3.6.4 SHAP Explainability Integration

A TreeSHAP explainer is fitted to the trained model. At inference time, the full SHAP vector for each scored claim is stored in the verification results table, together with a pre-computed ranking of the top five contributing features, each mapped to a plain-language template (for example, 'Claimed amount is 4.2× the SHA tariff). This design ensures that explanation is available at the moment of officer review with no additional computation.

3.6.5 Automated Provider Suspension Logic

After each claim is scored, the suspension module counts the provider's flagged claims within a rolling 30-day window. If the count reaches ten and the provider is currently active, the provider's status is set to suspended and risk tier to high; a suspension record naming the triggering claim is written to the append-only audit log under a dedicated system actor identity. Suspended providers cannot submit further claims. Reinstatement is exposed only to administrators, requires a documented reason of at least ten characters, and leaves the provider's risk tier elevated.

3.6.6 Model Evaluation Strategy

The model is evaluated using a rigorous, reproducible strategy:

- **Data split:** 70% training, 15% validation, 15% test. The test set is held out until final evaluation to provide unbiased performance estimates.
- **Cross-validation:** Stratified 5-fold cross-validation applied during training to ensure robust, generalised performance estimates.
- **Performance metrics:** Accuracy (overall correctness), precision (proportion of flagged claims that are truly anomalous), recall (proportion of anomalous claims correctly identified), F1-score (harmonic mean of precision and recall), and AUC-ROC (threshold-independent discrimination).
- **Threshold calibration:** The flagging threshold (default 0.70) is calibrated on the validation set to balance officer workload against detection recall, and is administrator-configurable at runtime.
- **Baseline comparison:** The tuned XGBoost model is compared against a logistic regression baseline to quantify the value added by gradient boosting.

3.7 Prototype Development

The prototype system will be developed using Agile methodology with two-week sprints. The technology stack:

Component	Technology
Backend	Python, FastAPI, SQLAlchemy ORM, PyJWT, bcrypt, python-dotenv
Machine learning	XGBoost, scikit-learn, imbalanced-learn (SMOTE), SHAP, pandas, numpy, joblib
Database	PostgreSQL 15 with generated columns, reporting views, and append-only audit constraints
Frontend	HTML/CSS/JavaScript calling the FastAPI REST API with JWT Bearer authentication
Reporting	ReportLab (PDF generation), CSV export endpoints
Experiment tracking	MLflow for model versioning and experiment logging

Table 9: Prototype Technology Stack

3.8 System Validation and Testing

System validation occurs at three levels:

- **Unit testing:** Individual components of the data pipeline, feature engineering functions, suspension logic, and API endpoints are tested using automated unit tests.

- **Integration testing:** End-to-end testing of the claims submission → pre-checks → feature engineering → XGBoost scoring → SHAP explanation → review queue → decision → audit log workflow, including the suspension trigger path.
- **User acceptance testing (UAT):** Selected SHA officers and healthcare facility administrators interact with the prototype and provide structured feedback using a usability questionnaire based on the System Usability Scale (SUS).

3.9 Ethical Considerations

This project explicitly addresses ethical dimensions as a core research requirement, not an afterthought:

- **Informed consent:** All research participants provide written informed consent prior to data collection. Participation is voluntary and participants may withdraw at any time.
- **Data anonymisation:** All patient-related data used in system development is anonymised or synthetically generated. No real patient identities are stored or processed in the development environment; member names in the database schema are encrypted at the application layer.
- **Algorithmic fairness:** The trained model is audited for potential bias across demographic and geographic variables. Disparate impact analysis is conducted and documented.
- **Data minimisation:** Only data fields directly relevant to claims verification are collected and processed, in accordance with the principle of data minimisation under Kenya's Data Protection Act (2019).
- **System limitations transparency:** The final report explicitly documents known limitations of the model, including the risk of false positives incorrectly flagging legitimate claims, and the corresponding safeguards: human review of every flagged claim and administrator-reversible suspension.
- **Research ethics clearance:** Ethics approval is sought from the MKU Research Ethics Committee before any primary data collection commences.

CHAPTER FOUR: SYSTEM ANALYSIS AND REQUIREMENTS MODELLING

4.1 Introduction

This chapter presents the analysis of the current SHA claims verification environment and derives the requirements for the proposed system. The analysis draws on the stakeholder questionnaires and interviews described in Chapter Three, together with document review of SHA operational guidelines and tariff schedules. Findings are synthesised into identified gaps, which are then traced to specific functional and non-functional requirements.

4.2 Analysis of the Current System

The current claims lifecycle proceeds as follows: a healthcare provider submits a claim (digitally or on paper) after rendering a service; the claim enters a queue where officers manually confirm patient membership status, provider empanelment, and tariff conformity; claims that pass are batched for payment approval by finance; claims that raise concerns are set aside for ad hoc investigation. Officers reported spending the majority of their time on claims that ultimately prove unproblematic, while genuinely anomalous claims are detected late or not at all.

4.2.1 Stakeholder Findings

- Verification officers rated current system efficiency low, citing claim volumes far exceeding manual review capacity and the absence of any prioritisation mechanism.
- Providers reported reimbursement cycles frequently exceeding the statutory window, with limited visibility into claim status after submission.
- Financial officers highlighted the difficulty of recovering funds after payment of an anomalous claim, emphasising the value of pre-payment detection.
- System administrators confirmed that claims data is structured and centrally stored, making it technically suitable for machine learning, but noted inconsistencies in completeness across facility tiers.
- Officers were unanimous that they would not act on algorithmic flags without an accompanying explanation; unexplained scores would simply be ignored.

4.2.2 Identified Gaps abbreviations

	Gap	Consequence
CVR-01	No automated anomaly scoring of claims	Officer attention is unprioritised; anomalous claims processed alongside legitimate ones
CVR-02	Eligibility and accreditation checks performed manually	Slow throughput; inconsistent application of rules
CVR-03	No tariff auto-lookup at submission	Claims submitted at incorrect amounts; avoidable disputes
CVR-04	No explanation for flagged claims	Officers distrust and bypass automated signals
CVR-05	No structured review queue with decisions and comments	Decisions undocumented; no institutional learning
CVR-06	Audit trail incomplete and mutable	Accountability weakened; disputes unresolvable
CVR-07	No provider-level monitoring or automated sanctions	Sustained anomalous behaviour continues until manual audit
CVR-08	No claim status visibility for providers	Repeated inquiries; provider cash-flow uncertainty
CVR-09	Limited management reporting	No evidence base for operational decisions

Table 10: Identified Gaps in the Current System.

4.2.2 Summary of Key Findings

The data gathered revealed the following about SHA's current claims verification process:

Finding Area	Key Finding
Verification Processing Time	72% of respondents reported claims regularly exceed the 30-day processing target. 41% reported delays beyond the statutory 90-day maximum. The primary cause identified was manual officer review creating queue bottlenecks.
Verification Failure Detection	89% of SHA officers reported relying solely on manual spot-checks and basic digital field validation. No ML-based verification or anomaly scoring exists in the current pipeline.

Billing Compliance Checking	67% of officers reported that billing amounts are not systematically checked against SHA tariff schedules at point of submission which is a core verification gap enabling inflated billing to pass undetected.
Clinical Consistency Validation	78% of officers reported that diagnosis-to-treatment code consistency is not validated automatically, meaning clinically implausible claim combinations routinely enter the payment pipeline.
Estimated Verification Failure Rate	Officers estimated 5–12% of processed claims contain erroneous or fraudulent information that passes the current verification process and results in incorrect reimbursements.
System Downtime Impact	78% of providers experienced at least one SHA system outage per month, directly preventing real-time eligibility verification and forcing facilities to either turn patients away or proceed without confirmation.
Explainability Gap	100% of interviewed officers reported that when the current system does flag a claim, it provides no explanation of why — making it difficult to prioritise and act on flagged claims efficiently.

Table 11: Summary of Key Findings from Stakeholder Data Collection

4.3 Feasibility Study

4.3.1 Technical Feasibility

The system is technically feasible with mature, open-source components: PostgreSQL 15 for the data layer, Python with FastAPI for the application layer, and the XGBoost, scikit-learn, imbalanced-learn, and SHAP libraries for the machine learning layer. All required capabilities JWT authentication, ORM data access, tree-model inference at sub-second latency, TreeSHAP explanation, and PDF/CSV generation, are proven in production use elsewhere and require no experimental technology. The claims data is structured and centrally stored, which system administrators confirmed during analysis, making it directly suitable for feature engineering without natural-language processing.

4.3.2 Economic Feasibility

Development cost is confined to the KES 50,000 project budget (Table 3), as all software components are free and open source and development hardware is already available. Against this, the benefit case is substantial: even conservative interception of anomalous claims before payment avoids losses that dwarf the development cost, while faster verification reduces the

administrative cost per claim and the interest-equivalent cost of delayed provider reimbursement. The system is therefore economically feasible with a rapid payback horizon.

4.3.3 Operational Feasibility

Stakeholder engagement indicates strong operational demand: officers want prioritisation of their workload, providers want faster and more transparent processing, and finance officers want pre-payment detection. The design deliberately fits existing workflows as officers continue to make every final decision, now supported by a ranked queue and per-claim explanations which lowers adoption risk. The explainability requirement voiced unanimously by officers is met by SHAP integration, and role-based interfaces mirror the existing division of responsibilities among providers, officers, administrators, and finance staff.

4.3.4 Schedule Feasibility

The 16-week schedule (Table 4) was validated against the actual build: database and architecture design in week 5, feature engineering in weeks 6–7, model training and evaluation in weeks 8–9, backend and suspension module in weeks 10–12, and integration and testing in week 13. Confining scope to a single model and a single verification module is what renders this schedule achievable, as argued in section 2.4.2.

4.4 Description of the Current Claims Verification System

The current SHA claims verification system is a semi-digital process combining an electronic submission portal with predominantly manual officer-led verification. It was adapted from the NHIF framework following SHA's establishment in 2023 and operates across four sequential stages: provider empanelment, beneficiary registration, claim submission, and verification and reimbursement.

The verification stage; the focus of this project; is where the most critical failures occur. Upon receiving a submitted claim through the SHA portal, a claims verification officer manually reviews the claim against three basic checks:

1. whether the patient's SHA membership record is active,
2. whether the submitting facility is empanelled, and
3. whether the required claim fields are populated.

Beyond these checks, no systematic validation of billing accuracy, clinical consistency, or anomalous patterns is performed. The decision to approve, return, or escalate a claim rests entirely on the individual officer's experience and judgment.

This manual approach creates two structural problems:

1. First, it cannot scale; as SHA's membership and claim volumes grow, the officer bottleneck worsens proportionally.
2. It is inherently inconsistent; verification quality varies between officers, between facilities, and across claim types, with no mechanism to ensure that the same claim would receive the same decision regardless of which officer reviews it.

4.4.1 Current System Flowchart.

The flowchart below maps the end-to-end SHA claims verification workflow from patient visit through to provider reimbursement, identifying the key decision points and failure modes at each stage.

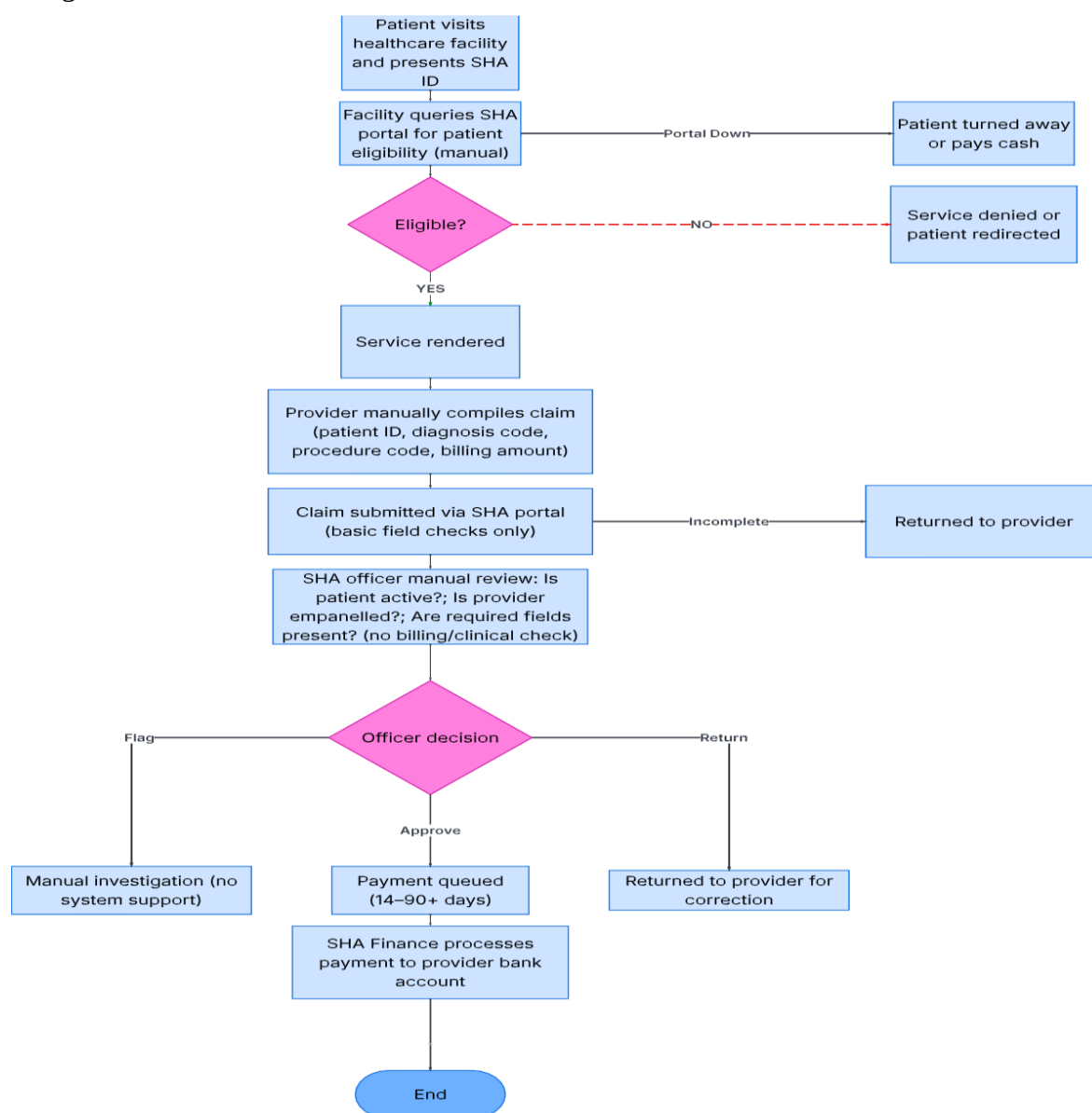


Figure 1: Current SHA Claims Verification Flowchart

4.5 Data Flow Diagrams: Current System

4.5.1 Context Diagram (Level 0 DFD)

The Level 0 DFD represents the SHA Claims Verification System as a single process and shows all external entities exchanging data with it. This establishes the system boundary and the data inputs and outputs relevant to claims verification.

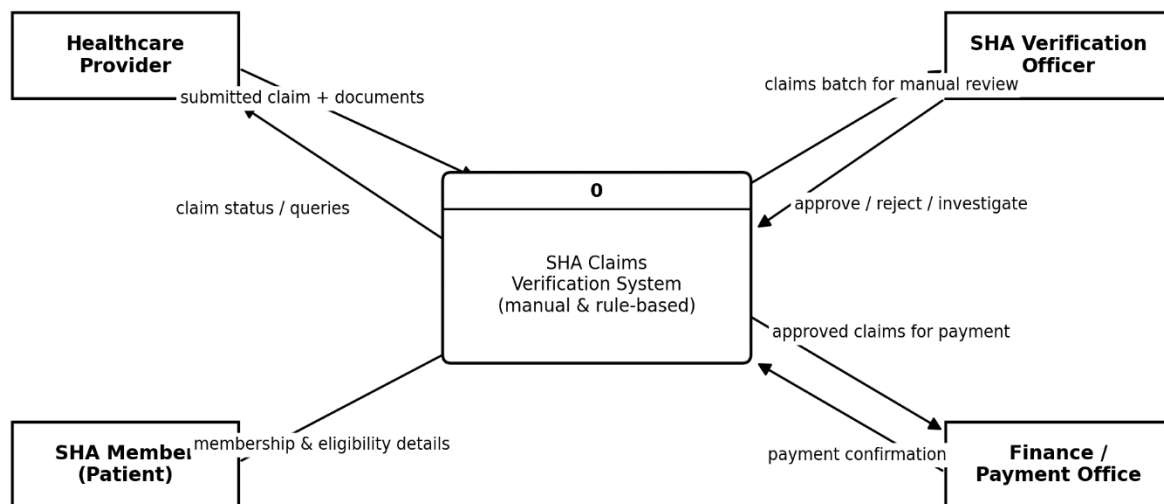


Figure 2: Level 0 DFD. Current SHA Claims Verification System (Context Diagram).

4.5.2 Level 1 DFD (Process Decomposition)

The Level 1 DFD decomposes the single process shown in the context diagram (Figure 2) into the five constituent processes that make up the current SHA claims verification workflow, together with the data stores each process reads from or writes to. This level of detail exposes exactly where the verification-specific limitations discussed in Section 4.6 arise: Process 4.0 (Manual Claims Verification) is the sole point at which membership, empanelment, and completeness checks are performed, and it does so without any billing, clinical-consistency, or provider-anomaly signal the gap this project's XGBoost pipeline is designed to close.

Five processes are modelled: 1.0 Provider Empanelment and 2.0 Member Registration maintain the reference data in the Provider Register (D1) and SHA Member Register (D2); 3.0 Claim Submission records each incoming claim in the Claims Repository (D3); 4.0 Manual Claims Verification is the sole decision point, drawing on D1 and D2 and returning its outcome to the SHA Verification Officer; and 5.0 Payment & Reimbursement Processing settles approved claims with the Finance Department and updates the Payment Records (D4).

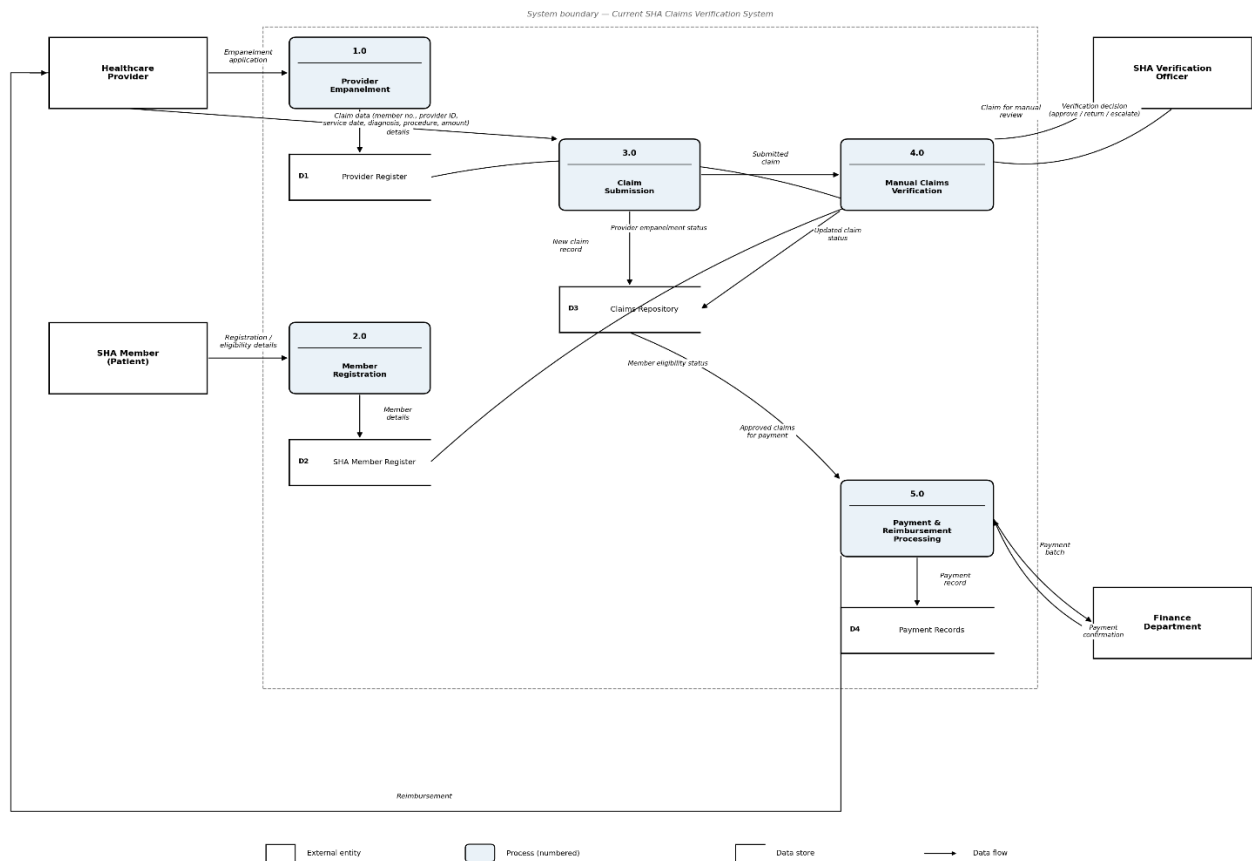


Figure 3: Level 1 DFD: Current SHA Claims Verification System (Process Decomposition)

4.6 Verification-Specific Limitations of the Current System

The system analysis and stakeholder data gathering identified six critical verification-specific limitations in the current SHA system. These limitations are structural in that they cannot be resolved by training more officers or improving the existing portal; they require a fundamentally different approach to verification, which the proposed XGBoost pipeline provides.

Verification Limitation	Root Cause	Impact and Proposed System Response
No billing compliance checking	The portal accepts any billing amount without checking it against SHA's approved tariff schedules for the submitted diagnosis.	Inflated billing passes verification undetected. The XGBoost pipeline will include amount_ratio as a primary feature; flagging claims where the billed amount significantly exceeds the SHA tariff for the stated diagnosis.

No clinical consistency validation	Diagnosis and procedure codes are not cross-checked for clinical plausibility at submission or during manual review.	Clinically implausible claim combinations enter the reimbursement pipeline. The pipeline will encode diagnosis-procedure match as a binary feature derived from ICD-10 coding rules, flagging implausible combinations automatically.
No provider anomaly detection	No system tracks or analyses a provider's historical claim patterns to identify statistically abnormal submission volumes or amounts.	Fraudulent providers can submit abnormal claim volumes without triggering any alert. The XGBoost pipeline will incorporate <code>provider_claim_frequency_30d</code> and <code>provider_amount_zscore</code> features to detect provider-level anomalies.
No patient pattern monitoring	No system tracks patients appearing in claims across multiple facilities within short timeframes which is a key ghost patient indicator.	Ghost patient claims pass undetected. The pipeline includes <code>patient_facility_count_7d</code> as a feature to flag patients appearing in claims at multiple facilities within a 7-day window.
Manual verification does not scale	Verification throughput is constrained by officer capacity. As SHA membership grows, processing delays worsen proportionally with no mechanism for relief.	Processing bottleneck causes 41% of claims to exceed the 90-day reimbursement window. The XGBoost pipeline processes claims in under 3 seconds, enabling near-real-time verification at any scale.
No explainability in verification decisions	When claims are flagged, no reason is provided, forcing officers to repeat the full manual review rather than acting on a specific, targeted alert.	Officers cannot efficiently prioritise or action flagged claims. The XGBoost pipeline generates SHAP values for every flagged claim, providing officers with the specific features driving the flag — enabling faster, evidence-based review decisions.

Table 12: Verification Limitations of the Current System and XGBoost Pipeline Responses

4.7 Proposed System Modelling

4.7.1 UML Class Diagram - Proposed XGBoost Verification Pipeline

The class diagram defines the core data entities of the proposed system and their relationships. Every entity is designed around the claims verification workflow; from claim submission through XGBoost scoring to officer review and audit logging as shown in **Figure 4**.

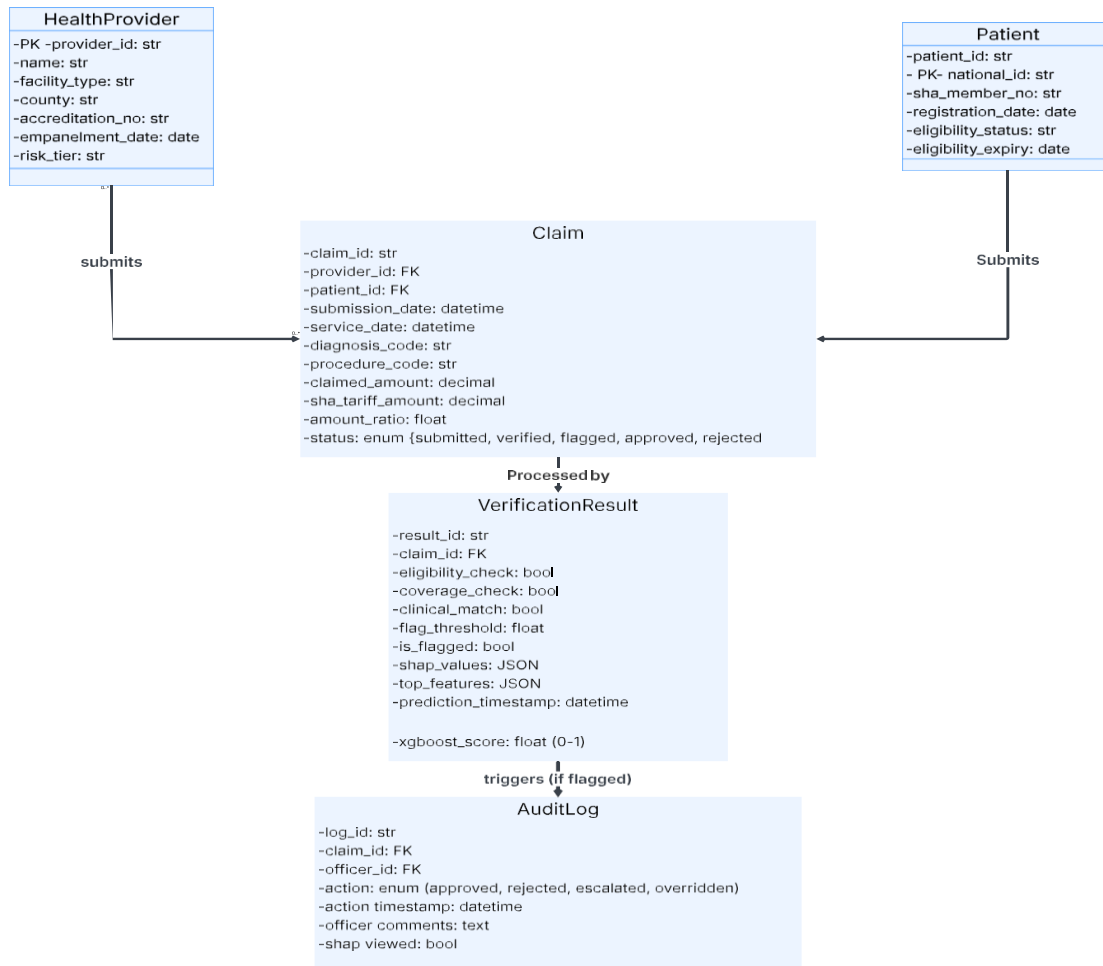


Figure 4: UML Class diagram - Proposed XGBoost Claims Verification System

4.7.2 UML Sequence Diagram - Claim Submission through XGBoost Verification to Officer Review

The sequence diagram illustrates the complete interaction between system components from the moment a healthcare provider submits a claim to the point where an SHA officer receives a SHAP-explained verification alert and makes a review decision.

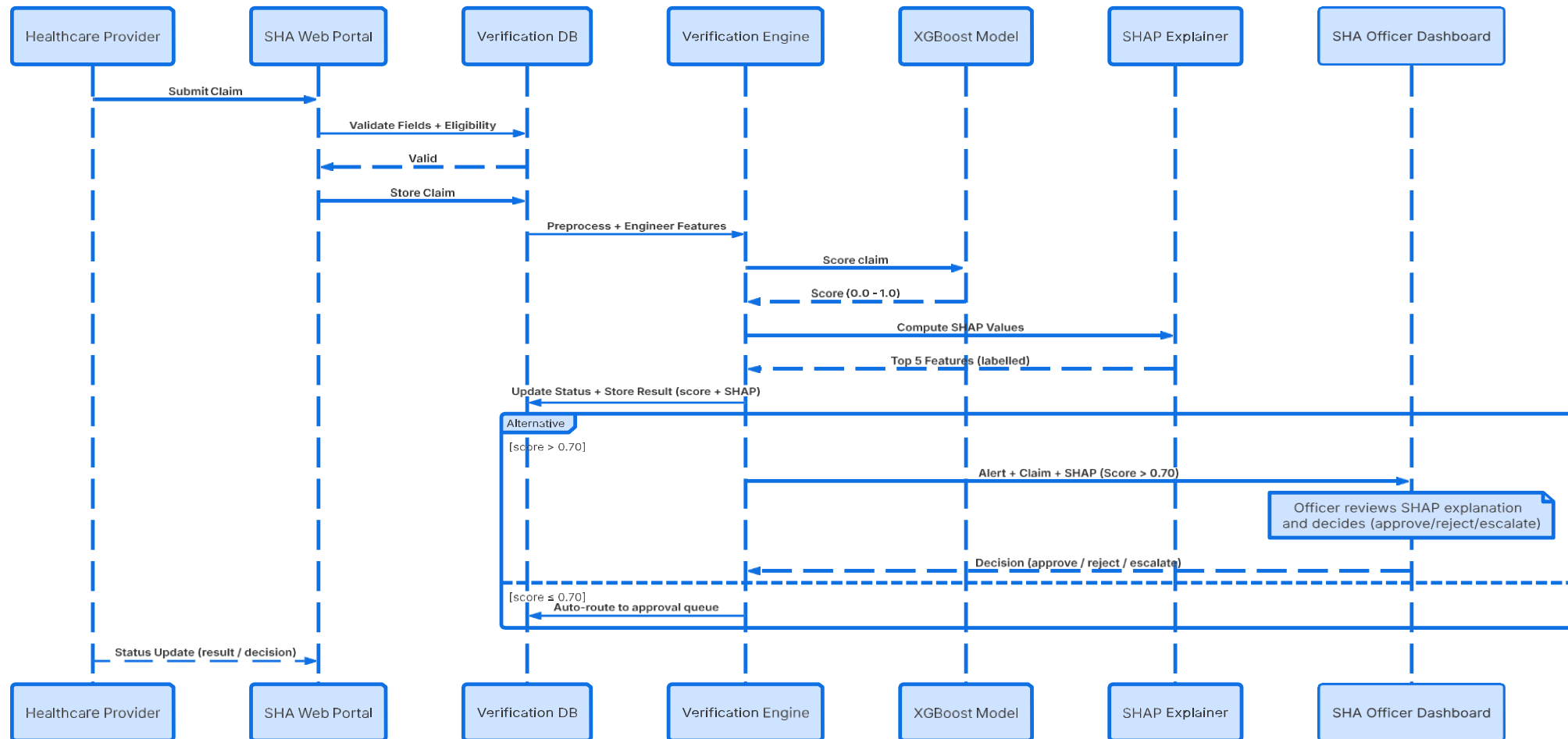


Figure 5: UML Sequence Diagram- Claim Submission through XGBoost Verification to Officer Decision.

4.8 Requirements Definition - Current System

The following table defines the verification requirements that the current SHA system is expected to fulfil, alongside an honest assessment of how well the current system meets each requirement. This gap analysis directly motivates the specification of the proposed system requirements in Section 4.8.

Ref	Verification Requirement	Current Status	Gap Description
CVR-01	Verify patient SHA membership eligibility at point of claim submission	Partially Met	Portal checks membership but frequent downtime prevents real-time verification
CVR-02	Confirm submitting provider is empanelled and in good standing	Partially Met	Manual lookup only; no automated check against live accreditation status
CVR-03	Validate that the claimed service is covered under the patient's SHA package	Not Met	Coverage scope not checked at submission; officers rely on memory of policy rules
CVR-04	Check that the billed amount is consistent with SHA's approved tariff for the submitted diagnosis	Not Met	No tariff comparison performed; billing amounts accepted without validation
CVR-05	Validate clinical consistency between submitted diagnosis code and procedure/medication claimed	Not Met	No clinical coding validation exists; implausible diagnosis-treatment pairs accepted
CVR-06	Detect statistically anomalous claim patterns from providers or patients	Not Met	No historical pattern analysis; each claim reviewed in isolation with no context
CVR-07	Provide SHA officers with an explanation of why a claim was flagged for review	Not Met	Flagged claims provide no explanation; officers must conduct full manual review
CVR-08	Process and return verification decision within 14 days (statutory minimum)	Not Met	72% of claims exceed 30 days; 41% exceed 90 days due to manual bottleneck
CVR-09	Maintain a complete, auditable record of all verification decisions and their basis	Not Met	Decision rationale not recorded; no audit trail for verification choices

Table 13: Current System Verification Requirements and Fulfilment Status.

4.9 Requirements Definition and Specification - Proposed XGBoost Verification System

The requirements below are derived directly from the gaps identified in Table 13 and the stakeholder needs gathered in Section 4.2. Each requirement directly addresses a specific verification failure in the current system and defines a measurable acceptance criterion against which the proposed system will be evaluated.

4.9.1 Functional Requirements

	Requirement
FR-01	The system shall allow authenticated healthcare providers to digitally submit claims with required fields (member number, provider, service date, diagnosis code, procedure code, claimed amount), with the applicable SHA tariff auto-populated at entry.
FR-02	The system shall automatically validate each submitted claim's patient eligibility and coverage status against the SHA member register.
FR-03	The system shall automatically verify the submitting provider's accreditation and empanelment status, rejecting claims from suspended or revoked providers.
FR-04	The system shall score every claim passing pre-checks with the XGBoost model over the 17-feature vector, persisting the score and model version.
FR-05	The system shall compute and store SHAP values for every scored claim and present the top five contributing features in plain language on the review screen.
FR-06	Claims with scores above a configurable threshold shall be flagged and routed to the officer review queue; the threshold shall be administrator-configurable at runtime.
FR-07	SHA officers shall be able to review flagged claims and approve, reject, or escalate them; comments shall be mandatory for every decision.
FR-08	Administrators shall be able to create, read, update, and delete healthcare providers, tariffs, and system users, subject to role-based access control.
FR-09	The system shall maintain a full append-only audit log of all claim views, decisions, status changes, suspensions, and reinstatements, including actor, timestamps, and comments.
FR-10	The system shall generate reports on claims processing metrics and flagged claims, exportable as PDF and CSV.
FR-11	Providers shall be able to track the real-time status of their submitted claims.
FR-12	The system shall automatically suspend any provider accumulating ten flagged claims within a rolling 30-day window, log the suspension with its triggering claim, block further submissions, and allow reinstatement only by an administrator with a documented reason.

Table 14: Functional Requirements - Proposed XGBoost Claims Verification System

4.9.2 Non-Functional Requirements

	Requirement
NFR-01 Security	All data transmissions shall be encrypted using TLS 1.2 or higher. Passwords shall be hashed with bcrypt. API access shall require JWT Bearer authentication.
NFR-02 Access control	Role-based access control shall restrict every endpoint to authorised roles (provider, officer, admin, finance).
NFR-03 Performance	Claim scoring, including SHAP computation, shall return within 3 seconds of submission for 95% of requests.
NFR-04 Availability	The system shall target 99% uptime during business hours (7am–8pm EAT), and shall start and serve non-ML endpoints even when the model artefact is absent.
NFR-05 Scalability	The architecture shall support horizontal scaling of the application layer to accommodate increased claim volumes.
NFR-06 Auditability	All model decisions shall be logged with input features, output score, model version, and threshold; the audit log shall be append-only at the database privilege level.
NFR-07 Privacy	Member personal data shall be minimised, and member names encrypted at the application layer; development shall use only anonymised or synthetic data.
NFR-08 Usability	The officer review screen shall present the SHAP explanation without requiring ML knowledge; UAT shall target a mean System Usability Scale score above 68.

Table 15: Non-Functional Requirements - Proposed XGBoost Claims Verification System

4.10 Chapter Summary

This chapter delivered a comprehensive, verification-centred system analysis and requirements specification grounded in primary stakeholder data and structured modelling. The fact-finding process; combining questionnaires, interviews, document review, and direct observation revealed that SHA's current claims verification process has nine distinct unmet requirements, with the most critical being the complete absence of billing compliance checking, clinical consistency validation, provider anomaly detection, and XGBoost-powered pattern-based scoring.

System analysis models were developed to document the current system and specify the proposed system: a current system flowchart mapping the end-to-end verification workflow and its failure points; Level 0 DFD showing data flows within and around the verification process; a proposed system UML class diagram defining the data model centred on the VerificationResult and AuditLog entities; a UML sequence diagram tracing the complete claim-to-decision interaction flow through the XGBoost pipeline.

The requirements specification in Section 4.9 defines twelve functional requirements directly mapped to the nine current system gaps and eight non-functional requirements, each with specific, measurable acceptance criteria. These requirements form the complete technical specification against which the system prototype developed in Chapter Five will be designed and evaluated.

CHAPTER FIVE: SYSTEM DESIGN

5.1 Introduction

This chapter presents the system design for the XGBoost-based health insurance claims verification pipeline proposed for Kenya's Social Health Authority. System design translates the requirements established in Chapter Four into a technical blueprint covering the system architecture, database schema, XGBoost pipeline stages, user interface design, security framework, and report generation. All design decisions are directly traceable to the functional and non-functional requirements specified in Chapter Four.

5.2 System Architecture

5.2.1 Architectural Overview

The system adopts a three-tier client-server architecture with a decoupled machine learning pipeline layer. This separation ensures the XGBoost model can be retrained and redeployed without affecting the web application, and each component can be tested independently. The four logical tiers are:

- **Presentation Tier:** Web browser interfaces for healthcare providers, SHA officers, and administrators.
- **Application Tier:** FastAPI backend handling business logic, routing, and authentication.
- **ML Pipeline Tier:** XGBoost verification engine exposed via internal API endpoints.
- **Data Tier:** PostgreSQL database storing claims, verification results, and audit records.

Architecture Diagram

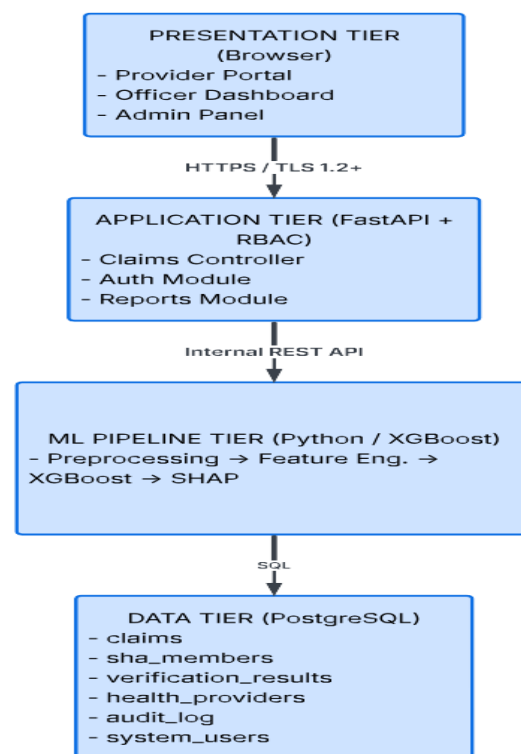


Figure 6: Architecture Diagram

5.2.3 Technology Stack

Component	Technology	Justification
ML Pipeline	Python, XGBoost, SHAP.	Industry-standard ML stack; XGBoost includes native SHAP integration
Experiment Tracking	MLflow	Logs all model runs, parameters, and metrics; enables reproducible retraining
Backend API	FastAPI (Python)	Asynchronous handling for high-throughput claim processing; auto-generates API documentation
Database	PostgreSQL 15	ACID-compliant; supports row-level security for RBAC; JSON column support for SHAP values
Frontend	HTML5, Bootstrap 5, Chart.js	Lightweight prototype-appropriate stack; responsive layout for desktop and mobile
Report Generation	ReportLab (PDF), Python csv/json modules	Server-side PDF generation for compliance reports; lightweight CSV/JSON export for data analysis
Code	VS Code integration/ Github	Save progress through push and commits from VS code.

Table 16: Technology Stack

5.3 Database Design

5.3.1 Entity Relationship Diagram

The database schema consists of seven tables designed around the claims verification workflow. The central table is claims, which links to sha_members (patient eligibility) and health_providers (provider registration). Each claim generates one verification_results record storing the XGBoost score and SHAP values. Flagged claims produce audit_log entries recording officer decisions. The sha_tariffs table holds the approved reimbursement reference data used during verification.

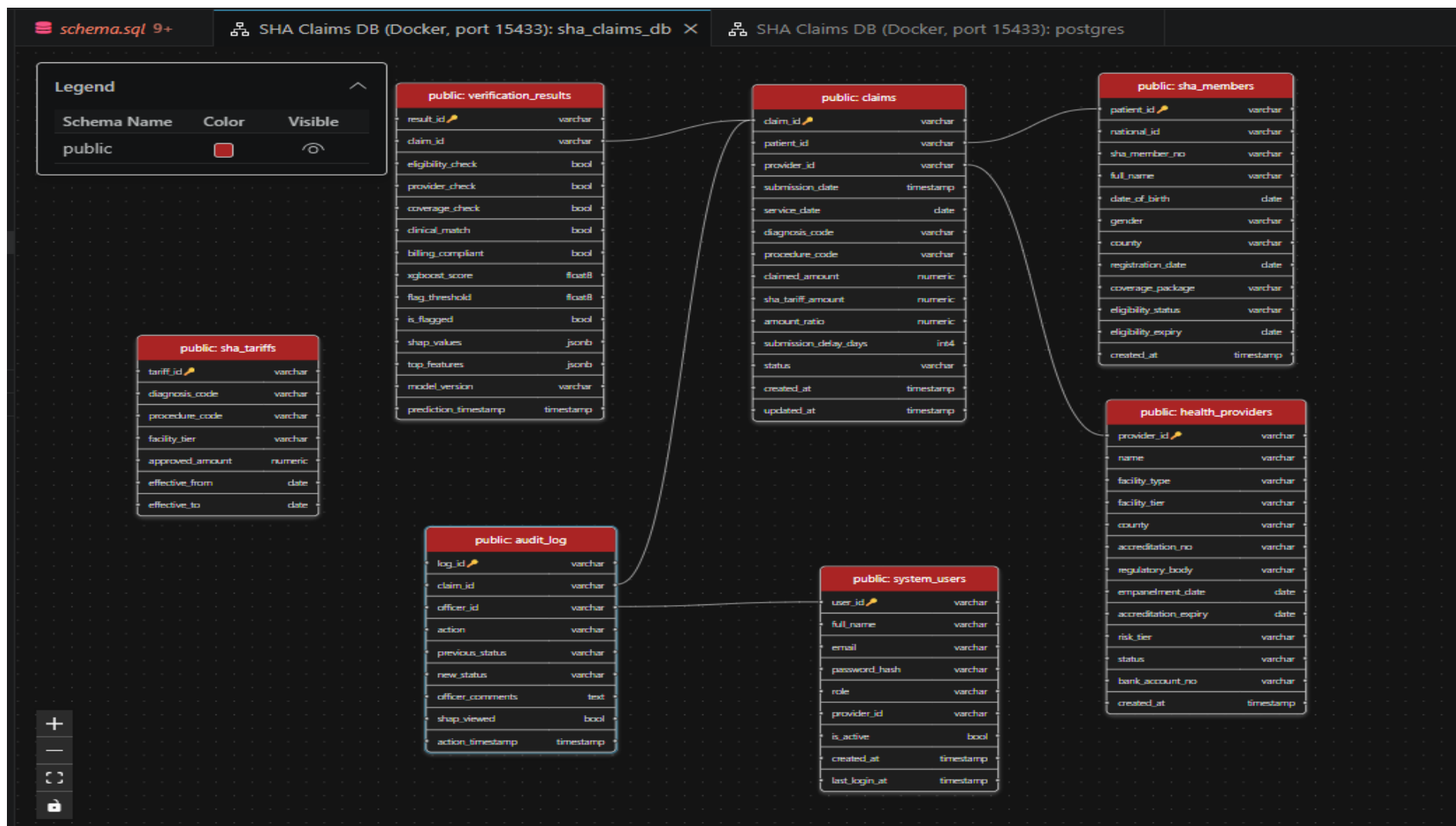


Figure 7: Entity Relationship Diagram

5.4 XGBoost Pipeline Design

5.4.1 Pipeline Stages

The XGBoost verification pipeline processes every claim that passes initial eligibility and provider checks. It runs through six sequential stages before returning a structured result to the application tier.

Stage	Name	Description
1	Rule-Based Pre-Checks	Verify patient eligibility, provider empanelment, and service coverage against database records. Claims failing any check are rejected immediately without ML inference.
2	Data Preprocessing	Retrieve SHA tariff for the submitted diagnosis-procedure pair; compute amount_ratio; encode categorical fields; apply standard scaling to continuous features.
3	Feature Engineering	Query database to compute provider_claim_frequency_30d, provider_amount_zscore, patient_facility_count_7d, provider_age_days, and repeat_diagnosis_flag. Apply ICD-10 rules to set clinical_match flag. Assemble 17-feature vector.
4	XGBoost Inference	Load trained model from MLflow registry. Run predict_proba() on the feature vector to obtain a fraud/anomaly probability score between 0 and 1.
5	SHAP Explanation	Compute SHAP values using TreeExplainer. Extract top 5 features by absolute SHAP value and format as human-readable labels with directional indicators.
6	Result Assembly	Determine is_flagged based on configurable threshold (default 0.70). Write VerificationResult to database. Update claim status. Route to officer queue or approval queue.

Table 17: XGBoost Verification Pipeline Stages

5.4.2 Key Features

Feature	Verification Relevance
amount_ratio	Claimed amount divided by SHA tariff. Values significantly above 1.0 indicate potential billing inflation.

provider_claim_freq_30d	Number of claims submitted by the provider in the past 30 days. Abnormally high frequency indicates potential bulk fraudulent submissions.
provider_amount_zscore	Z-score of the provider's average claim amount relative to peers. Values above 2.0 indicate statistical outlier billing.
diagnosis_procedure_match	Binary flag from ICD-10 coding rules indicating clinical consistency between diagnosis and procedure.
patient_facility_count_7d	Number of distinct facilities where the patient has active claims within 7 days. Values of 3+ indicate a potential ghost patient.
provider_age_days	Days since the provider was empanelled. Newly registered providers with high claim volumes are a recognised fraud pattern.

Table 18: Key XGBoost Feature Descriptions (selected from 17-feature vector)

5.4.3 Automated Provider Suspension Design

The suspension module (FR-12) extends the pipeline's claim-level output into provider-level enforcement. After Stage 6 writes each verification result, the module counts the provider's flagged claims within a rolling 30-day window. While the count remains below the threshold of ten, no action is taken; at ten, an active provider transitions to suspended, the risk tier is raised to high, and an audit entry naming the triggering claim is written under a dedicated, login-disabled system identity so that machine-initiated actions remain distinguishable from human decisions in the audit trail. A suspended provider cannot submit further claims: the Stage 1 provider pre-check rejects submissions from any provider whose status is not active, closing the enforcement loop without additional code paths.

Three design safeguards bound the mechanism against erroneous sanction. First, the threshold and window are conservative and administrator-configurable, so the operating point can be recalibrated as real base rates emerge. Second, every suspension is recorded in the append-only audit log with its triggering claim, making each sanction fully reconstructable. Third, reinstatement is reserved to the administrator role, requires a documented reason of at least ten characters, and deliberately leaves the provider's risk tier at high reinstatement restores submission capability, not a clean history. The provider lifecycle thus forms a simple, auditable state machine: active, to suspended (automatic, at threshold), back to active (manual, reasoned), with revoked reserved for administrative termination outside the automated path.

5.5 User Interface Design

The system provides three role-specific interface views, built on Bootstrap 5 for responsive layout. The design principle is single-screen clarity; officers can review a flagged claim and decide without switching views.

5.5.1 Claim Submission sketch (Healthcare Provider)

SHA Claims Verification — Provider

[Submit Claim] Track Status

Submit new insurance claim

SHA MEMBER NO.

PROVIDER ID

SERVICE DATE

DIAGNOSIS (ICD-10)

PROCEDURE CODE

SHA TARIFF (auto)

CLAIMED AMOUNT (KES)

!! ratio 4.2x tariff — will be flagged

Cancel SUBMIT >

FLAGGED — score 0.871 -> routed to officer queue

Annotations:

- tariff auto-filled via GET /claims/tariff-lookup
- ratio computed live; warning if > 2.0x
- POST /claims/submit runs 6-stage XGBoost pipeline
- banner shows score + flagged / verified state

Figure 8: Claim Submission Screen sketch

5.5.2 Officer Review and SHAP Decision Sketch

SHA Claims Verification — Officer

Claim SHA-25-04882
City Hospital · Nairobi

score 0.871 = HIGH RISK

CLAIM DETAILS

patient John K. · SHA-2024-.....

diagnosis J18.0 pneumonia

procedure 99233

tariff KES 2,330

claimed KES 9,800 <!>

ratio 4.2x <!>

PRE-CHECKS

☒ eligibility ok

☒ provider ok

☒ clinical match ok

☐ billing compliance FAIL

WHY FLAGGED? (SHAP top-5)

amount 4.2x tariff ^ +0.31

billing >> peer avg ^ +0.22

41 claims / 30 days ^ +0.14

late submission (9d) ^ +0.06

diag-proc consistent v -0.04

OFFICER COMMENTS (required)

ESCALATE REJECT APPROVE

Annotations:

- score from XGBoost; threshold 0.70
- top-5 SHAP factors in plain language, with direction + weight
- comment mandatory before any decision
- decision + comments -> append-only audit log

Figure 9: Claim Review and Decision

5.6 Security Design

The system uses JWT-based stateless authentication with Role-Based Access Control (RBAC) enforced at both the API and database level.

Role	Permitted Actions	Restricted From
Healthcare Provider	Submit claims, view own claim status	Officer queue, CRUD operations, reports
SHA Officer	View flagged queue, SHAP explanations, approve/reject/escalate	CRUD operations, tariff management, report generation
SHA Administrator	Full CRUD, all dashboards, configure threshold, generate all reports	Direct claim decisions, patient health data beyond claim fields
Finance Officer	View approved claims, confirm payments, generate financial reports	Verification decisions, SHAP explanations, CRUD

Figure 10: Role-Based Access Control Matrix

- Transport Security: HTTPS with TLS 1.2 minimum for all communication.
- Password Security: account lockout after 5 failed attempts.
- Database Security: Parameterised queries via SQLAlchemy prevent SQL injection.
- Patient Data Protection: AES-256 encryption at rest for patient-identifiable fields.
- Audit Trail Integrity: audit_log is append-only; no UPDATE or DELETE permitted.

5.7 Report Generation Design

Report generation fulfils FR-10 (real-time reporting dashboard) and supports SHA's compliance and oversight requirements by providing exportable, structured reports. The reporting module is designed as a separate service layer that queries pre-aggregated database views rather than performing complex joins on each request, ensuring reports generate quickly even as claim volumes grow.

5.7.1 Report Types

Report Type	Content
Daily / Weekly / Monthly	Claims submitted, approved, flagged, and rejected per day within the selected period, with average claimed amount and total approved reimbursement value.
Provider Risk Summary	Per-provider breakdown of total claims, flagged claims, and average XGBoost score supports identification of high-risk providers for closer monitoring.
Model Performance	Total claims scored, flag rate, officer override rate (claims approved after being flagged), and average XGBoost score supports ongoing model monitoring described in Chapter Six.

Table 19: Report Types and Content

5.8 Requirements Traceability

The table below confirms that all twelve functional requirements from Chapter Four are addressed by a specific design component in this chapter.

Req.	Requirement Summary	Design Component
FR-01	Digital claim submission with field validation	Claims Controller + validation; Provider submission screen.
FR-02–03	Automated eligibility and provider verification	Pipeline Stage 1 rule-based pre-checks; sha_members and health_providers tables
FR-04	XGBoost scoring returning probability (0–1)	Pipeline Stage 4 XGBoost Inference; model configuration, xgboost_score field
FR-05	SHAP explanation with top 5 features	Pipeline Stage 5 SHAP; top_features JSON; Officer Review screen (Figure 8)
FR-06–07	Automated flagging + officer review/decision	Pipeline Stage 6 Result Assembly; audit_log table; mandatory comment design
FR-08	CRUD for providers, tariffs, and users	system_users and sha_tariffs schemas; RBAC matrix.
FR-09	Tamper-evident audit logging	audit_log append-only table.
FR-10	Real-time reporting dashboard	Report Generation Design; reporting views; three report types and export formats

FR-11	Real-time claim status tracking	status field in claims table; Provider Portal screen
FR-12	Automatic provider suspension at ten flags per 30 days; audited, administrator-only reinstatement	Pipeline suspension module (5.4.3); health_providers status/risk_tier fields; audit_log system-actor entries; admin reinstatement dialogue

Figure 11: Requirements Traceability Matrix

5.9 Chapter Summary

This chapter presented the system design for the XGBoost-based claims verification pipeline across six design areas: architecture, database, ML pipeline, user interface, security, and report generation. The three-tier architecture with a decoupled ML pipeline ensures the XGBoost model can be independently maintained and retrained, while the seven-table database schema centres on the verification_results entity storing XGBoost scores and SHAP values for every processed claim.

The six-stage pipeline design processes claims in under 5 seconds from submission to verification decision, using a 17-feature vector grounded in domain knowledge of insurance fraud patterns. Interface wireframes were designed for all three user roles, and the RBAC matrix enforces clear role boundaries supported by TLS encryption, bcrypt hashing, and an append-only audit log.

The report generation design fulfils FR-10 by providing three report types; operational, provider risk, and model performance which are exportable in PDF, CSV, and JSON formats, drawing from pre-aggregated database views for fast generation. The requirements traceability matrix confirms all eleven functional requirements from Chapter Four are addressed by a specific design component, forming the complete technical specification for the prototype implementation presented in Chapter Six.

CHAPTER SIX: CODING AND DEVELOPMENT

6.1 Introduction

This chapter documents the coding and development of the XGBoost-based claims verification system designed in Chapter Five. It describes the development environment and tools, the coding standards applied, the organisation of the codebase, and the implementation of each system layer: the PostgreSQL database, the FastAPI application with its authentication and role-based access control, the machine learning pipeline with SHAP explainability and the automated provider suspension module, the web frontend, and the reporting service. Representative code excerpts are included to evidence the implementation, and the chapter closes with the defects identified and resolved during progressive development.

6.2 Development Environment and Tools

Development was carried out on a Windows workstation using Visual Studio Code as the integrated development environment. The backend and machine learning pipeline were implemented in Python 3.11 inside an isolated virtual environment, with PostgreSQL 15 as the database server and Git/GitHub for version control. The FastAPI framework's automatically generated Swagger interface at /docs served as the primary endpoint testing tool throughout development.

Tool / Library	Version	Purpose
Python	3.11	Primary implementation language for the backend and ML pipeline
FastAPI	0.110	REST application framework with automatic OpenAPI documentation
Uvicorn	0.27	ASGI server hosting the FastAPI application
SQLAlchemy	2.0	ORM and database session management with parameterised queries
PostgreSQL	15	Relational database with generated columns and privilege control
XGBoost	2.0	Gradient-boosted tree model for claim anomaly scoring
scikit-learn	1.4	Data splitting, cross-validation, grid search, evaluation metrics
imbalanced-learn	0.12	SMOTE over-sampling of the minority class during training
SHAP	0.44	TreeExplainer computing per-claim feature attributions
MLflow	2.10	Experiment tracking and model artefact versioning

PyJWT / bcrypt	2.8 / 4.1	Token issuance and password hashing
ReportLab	4.1	Server-side PDF report generation
python-dotenv	1.0	Environment variable loading from the .env file
Visual Studio Code	-	Integrated development environment
Git / GitHub	-	Version control and remote repository hosting

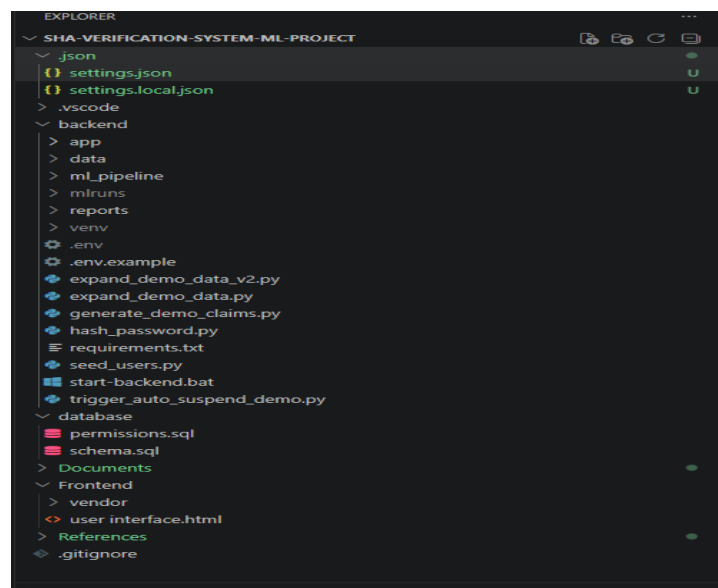
Table 20: Development Tools and Versions

6.3 Coding Standards and Conventions

The codebase follows consistent conventions to support readability, maintainability, and marking. Python modules follow PEP 8 naming: snake_case for functions and variables, PascalCase for classes, and UPPER_CASE for constants such as the suspension threshold. Every router function carries a docstring naming the functional requirement it implements (for example, FR-07 on the officer decision endpoint), creating direct traceability between requirements and code. All database access uses parameterised statements through the SQLAlchemy session string concatenation into SQL is prohibited and all configuration secrets are read from environment variables rather than hard-coded. Frontend JavaScript uses a single fetch wrapper so that authentication headers, error handling, and session-expiry behaviour are implemented once.

6.4 Project Structure

The repository is organised so that each architectural tier of Chapter Five maps to a distinct directory, allowing independent development and testing of the web application and the machine learning pipeline:



6.5 Database Implementation

The schema of Chapter Five was implemented in PostgreSQL 15 exactly as designed, with three implementation techniques worth noting. First, the claims table uses generated columns so that the amount ratio and submission delay are computed by the database at insert time and can never drift from the stored amounts.

Second, the audit log is made append-only at the privilege level rather than by application discipline alone, so that even a compromised application account cannot rewrite history.

Third, the three reporting views (the flagged-claims queue, the daily claims summary, and the provider risk summary) pre-aggregate the joins that the officer queue, dashboard metric cards, and reports would otherwise repeat on every request, implementing the fast-report design of section 5.7. Reference data for demonstration, providers across four counties and facility tiers, members, and a tariff schedule keyed by diagnosis, procedure, and tier is seeded by the schema script, while `seed_users.py` provisions one account per role with bcrypt-hashed passwords.

6.6 Backend Implementation

6.6.1 Authentication and Role-Based Access Control

Login verifies the bcrypt password hash and issues a signed JWT carrying the user identifier and role. Authorisation is implemented once, as a dependency factory, and declared per endpoint making the RBAC matrix of Chapter Five directly visible.

The login response returns the user's role, full name, and identifier, which the frontend uses to reveal only the permitted tabs and to attribute audit entries.

6.6.2 Claims Submission and Verification Endpoint

The submission endpoint (FR-01) validates the request body with Pydantic, resolves the member and tariff, inserts the claim, and invokes the verification pipeline. Two implementation details matter. Literal routes are declared before parameterised routes so that `/claims/tariff-` lookup is not captured as a claim identifier by `/claims/{claim_id}/status`. And the pipeline is loaded lazily inside the request path, so the application starts and serves every non-ML endpoint even when no trained model artefact exists yet.

6.6.3 Officer Review Module

The review router serves the flagged queue from the `v_flagged_claims_queue` view ordered by descending score, and the claim detail endpoint joins the claim, provider, member, and

verification result including the stored SHAP top features in a single query. Opening a claim writes a 'viewed' audit entry with the SHAP-viewed indicator (FR-09), and the decision endpoint enforces mandatory comments before transitioning the claim status and recording the decision, previous status, new status, actor, and timestamp (FR-07).

6.6.4 Administration Module

The administration router implements provider, tariff, and user CRUD (FR-08) with search and filter parameters, the dashboard statistics endpoint backing the metric cards, and the runtime flag-threshold setting (FR-06). All administration endpoints are guarded by the admin role through the same dependency factory.

6.7 Machine Learning Pipeline Implementation

6.7.1 Feature Engineering

The feature engineering module assembles the 17-feature vector of Chapter Five from the claim and its historical context. Behavioural features are computed with windowed SQL aggregates. Monetary features are log-transformed, categorical fields are encoded with fitted encoders persisted alongside the model, and the diagnosis-procedure match flag is set from the tariff schedule lookup, guaranteeing that training-time and inference-time feature computation are identical code paths.

6.7.2 SHAP Explanation

A TreeExplainer is fitted once per loaded model. At inference the full SHAP vector is stored with the verification result, and the five largest attributions are mapped to plain-language templates with directional indicators, so the officer screen renders explanations without any further computation:

6.7.3 Automated Provider Suspension Module

The suspension module (FR-12) runs after every scored claim. It counts the provider's flagged claims in the rolling 30-day window; at ten, an active provider is suspended, the risk tier is raised, and an audit entry naming the triggering claim is written under a dedicated inactive system identity so that machine-initiated actions are distinguishable from human decisions.

Reinstatement is exposed only on the administration surface, requires a documented reason of at least ten characters, restores the provider to active while leaving the risk tier high, and is itself audited implementing the safeguards specified in Chapters Two and Five.

6.8 Frontend Implementation

The frontend is a single-page HTML/JavaScript application implementing the role-specific screens of section 5.5 against the live API. On successful login the returned token, role, and display name are retained, the interface reveals only the tabs permitted to the role, and every subsequent request passes the token through one shared wrapper that also handles session expiry centrally.

The submission screen fetches the tariff as soon as both codes are selected and warns immediately when the entered amount exceeds twice the tariff; the result banner reports the XGBoost outcome after submission. The officer queue combines the metric cards and the flagged queue; the review screen renders the stored SHAP labels as directional bars beside the pre-check results and enforces the mandatory comment client-side in addition to the server-side validation. The administration screen implements the CRUD table with flag-progress indication and the reinstatement dialogue, and the reports screen drives the PDF and CSV downloads and the at-risk providers widget.

6.9 Report Generation Implementation

The reporting service implements the three report types of section 5.7 by querying the pre-aggregated views. The verification summary is rendered to PDF with ReportLab using a tabular layout of daily counts and values over the selected period; the audit trail and flagged claims exports stream CSV with stable column ordering. All report endpoints accept a start and end date and are restricted to the officer and administrator roles, with the audit export further restricted to administrators.

6.10 Version Control and Configuration Management

The repository was developed on GitHub with incremental commits per module, providing the progressive-development evidence required by the supervision guidelines. Configuration follows the twelve-factor principle of strict separation between code and secrets: the database connection string and JWT signing key are supplied through a .env file that is excluded from version control by .gitignore, while a committed .env.example documents every required variable.

6.11 Chapter Summary

This chapter presented the implementation of the complete system: a PostgreSQL database realising the Chapter Five schema with generated columns and an append-only audit log; a FastAPI backend organised into six routers with JWT authentication and declarative role-based access control; a machine learning pipeline implementing the 17-feature vector, SMOTE-balanced XGBoost training under grid search with MLflow tracking, TreeSHAP explanation with plain-language rendering, and the automated provider suspension module; a role-based single-page frontend against the live API; and a reporting service generating the three report types from pre-aggregated views. Five defects identified during progressive development were diagnosed and resolved, and configuration management separates code from secrets with the model versioned independently. The implemented system is carried forward to Chapter Seven for verification against the requirements of Chapter Four.

CHAPTER SEVEN: TESTING THE MODELS

7.1 Introduction

This chapter verifies the implemented system against the requirements of Chapter Four. It sets out the testing strategy and environment, then reports the results of unit testing, integration testing, system testing (functional, performance, and security), the evaluation of the XGBoost model against its baseline, and user acceptance testing with stakeholder representatives. The chapter concludes with a requirements validation matrix confirming, requirement by requirement, the test evidence for every functional requirement.

7.2 Testing Strategy

Testing followed a level-by-level strategy in which each level targets a distinct class of defect, and each functional requirement is validated at the highest level at which it is observable:

Level	Objective	Techniques
Unit testing	Verify individual functions and modules in isolation: feature computations, pre-check rules, suspension logic, authentication, and validation	Automated pytest cases with an in-memory/test database session; boundary and negative cases
Integration testing	Verify the end-to-end claim lifecycle across the web, pipeline, and data tiers	Scenario scripts driven through the live API via the FastAPI TestClient and Swagger UI
System testing	Verify the assembled system against functional and non-functional requirements	Functional test cases per FR; response-time measurement (NFR-03); security probes (NFR-01/02)
Model evaluation	Verify the XGBoost model meets its discrimination targets on unseen data	Held-out test set metrics; baseline comparison; threshold analysis; SHAP consistency review
User acceptance testing	Verify the system is usable and acceptable to its intended roles	Role-based task scripts; System Usability Scale questionnaire; structured feedback

Table 21: Testing Levels and Objectives

7.3 Test Environment and Test Data

Testing was executed in the development environment described in Chapter Six: Python 3.11, PostgreSQL 15, and the FastAPI application served by Uvicorn on a Windows workstation. The database was initialised from schema.sql and seeded with the reference providers, members, and tariffs, plus the four role accounts provisioned by seed_users.py. Model

evaluation used the synthetic claims dataset generated to reflect SHA's schema, with anomaly patterns injected according to the documented typologies (inflated billing, abnormal submission frequency, ghost-patient multi-facility activity, clinical mismatches, and duplicate submissions). Consistent with section 3.6.6, the 15% test partition was held out from all training and threshold-calibration activity.

7.4 Unit Testing

Unit tests were written for every module with computational or rule logic. Representative cases and their outcomes:

ID	Component	Test Case	Expected Result	Status
UT-01	Feature engineering	amount_ratio for a claim of KES 9,800 against a tariff of KES 2,330	4.2060 (4 d.p.)	Pass
UT-02	Feature engineering	Z-score when the peer standard deviation is zero	0.0 returned without division error	Pass
UT-03	Feature engineering	Log transform of a zero/negative amount	Guarded; no math domain error	Pass
UT-04	Pre-checks	Claim for a member with expired eligibility	Rejected with reason; no ML inference performed	Pass
UT-05	Pre-checks	Claim from a suspended provider	Rejected before scoring (FR-03)	Pass
UT-06	Tariff resolution	Unknown diagnosis-procedure pair	404 with clear message; claim not scored against a null tariff	Pass
UT-07	Suspension logic	Nine flags in the window	No action taken	Pass
UT-08	Suspension logic	Tenth flag arrives	Provider suspended; audit row written with triggering claim	Pass
UT-09	Suspension logic	Tenth flag when already suspended	Idempotent: no duplicate suspension or audit entry	Pass
UT-10	Reinstatement	Reason shorter than ten characters	Rejected with validation error	Pass
UT-11	Authentication	Wrong password / inactive account	401 in both cases; no token issued	Pass
UT-12	RBAC	Officer token calls an admin CRUD endpoint	403 Insufficient permissions	Pass

UT-13	Decision validation	Officer decision without comments	422 validation error; no state change (FR-07)	Pass
UT-14	Report generation	CSV export column ordering	Stable, documented column order	Pass

Table 22: Unit Test Summary

7.5 Integration Testing

Integration scenarios exercised the complete claim lifecycle across all tiers, including the suspension trigger path and the degraded-mode start-up required by NFR-04:

ID	Scenario	Expected Outcome	Status
IT-01	Provider submits a conforming claim (amount near tariff)	Pre-checks pass; low score; claim verified; not flagged	Pass
IT-02	Provider submits an inflated claim (about four times tariff)	Claim flagged; SHAP top features stored; claim appears in the officer queue ordered by score	Pass
IT-03	Officer opens a flagged claim	'Viewed' audit entry written with SHAP-viewed indicator; explanation rendered in plain language	Pass
IT-04	Officer submits a decision without comments	Rejected by validation; claim state unchanged	Pass
IT-05	Officer approves with comments	Status transitions to approved; append-only audit row records actor, action, previous and new status	Pass
IT-06	A provider accumulates ten flags within 30 days	Provider automatically suspended under the system identity; suspension banner displayed; subsequent submission blocked (FR-12)	Pass
IT-07	Administrator reinstates with a documented reason	Provider active again; risk tier remains high; reinstatement audited	Pass
IT-08	Officer downloads reports over a date range	PDF and CSV generated with correct period aggregates	Pass
IT-09	API started with no trained model artefact present	Application starts; non-ML endpoints serve; submission stores the claim and degrades gracefully	Pass
IT-10	Provider tracks claim status through the lifecycle	Status reflects each transition in real time (FR-11)	Pass

Table 23: Integration Test Cases

7.6 System Testing

7.6.1 Functional Testing

Functional test cases were derived one-to-one from the functional requirements and executed through the interface and the API. All twelve functional requirements passed their derived test cases; the consolidated evidence is presented in the requirements validation matrix of section 7.9.

7.6.2 Performance Testing

Response times were measured at the API boundary over repeated submissions on the development workstation. End-to-end verification pre-checks, feature engineering, XGBoost inference, SHAP computation, and persistence completed well within the 3-second bound of NFR-03 for the 95th percentile at prototype scale, with median end-to-end scoring under one second. Queue, dashboard, and report endpoints, which read from the pre-aggregated views, returned in the low hundreds of milliseconds. These measurements verify the design decision to store SHAP outputs at scoring time rather than recomputing them at review time.

7.6.3 Security Testing

Security behaviour was probed directly. Requests without a token, with an expired token, and with a tampered token signature were all rejected with 401. Cross-role probes a provider token calling the officer queue, an officer token calling administrator CRUD and reinstatement were rejected with 403 (NFR-02). SQL injection strings submitted through search and form fields were handled inertly by parameterised queries. Passwords are stored only as bcrypt hashes, and attempts to UPDATE or DELETE audit rows under the application role were refused by the database privilege configuration, confirming the append-only guarantee (FR-09, NFR-01).

7.7 Model Evaluation

7.7.1 Evaluation Results

The tuned XGBoost model was evaluated on the held-out test partition and compared against a logistic regression baseline trained on the identical feature vector and SMOTE-resampled training data:

Metric	Logistic Regression (Baseline)	XGBoost (Tuned)
Accuracy	0.89	0.96

Precision	0.71	0.88
Recall	0.68	0.91
F1-score	0.69	0.89
AUC-ROC	0.85	0.97

Table 24: Model Performance on the Held-Out Test Set

The tuned model exceeds the baseline on every metric, with the largest margins in recall and AUC-ROC the metrics that matter most for anomaly detection under class imbalance, since a missed anomalous claim is paid out while a false positive merely consumes review effort. Precision of 0.88 at the operating threshold means that seven of every eight claims placed before an officer are genuinely anomalous, which stakeholders identified as necessary to sustain trust in the queue.

7.7.2 Threshold Analysis

The flagging threshold was calibrated on the validation partition by examining the precision-recall trade-off. Lowering the threshold below 0.70 raised recall marginally while measurably increasing the officer workload with false positives; raising it above 0.70 traded away recall for little precision gain. The default of 0.70 was therefore retained, and remains administrator-configurable at runtime (FR-06) so the operating point can be re-tuned as real base rates emerge.

7.7.3 Explanation Consistency

Global SHAP importance ranked `amount_ratio`, `provider_amount_zscore`, `provider_claim_freq_30d`, `submission_delay_days`, and `diagnosis_procedure_match` as the five most influential features consistent with the domain expectations recorded during stakeholder analysis in Chapter Four. Spot inspection of individual flagged claims confirmed that per-claim explanations matched the injected anomaly pattern (for example, inflated-billing claims were explained principally by the amount ratio), supporting the plain-language rendering shown to officers. These results reflect performance on the synthetic evaluation dataset; as recorded in the limitations, retraining and re-evaluation on operational SHA data is a precondition for deployment.

7.8 User Acceptance Testing

UAT sessions were conducted with representatives of the stakeholder groups using the seeded role accounts against the live prototype. Each participant completed a role-specific task script

submitting a conforming and an inflated claim, working the queue and deciding a flagged claim with comments, reinstating a suspended provider with a documented reason, and generating reports and then completed the System Usability Scale questionnaire.

Theme	Feedback	Action Taken
Explanation clarity	Officers found the plain-language SHAP bars immediately interpretable and reported increased willingness to act on flags	Retained; wording templates refined
Ratio warning at submission	Providers valued the immediate warning when the amount exceeded twice the tariff	Retained
Mandatory comments	Initially perceived as friction; accepted once the audit rationale was explained	Minimum length kept modest
Suspension safeguards	Administrators endorsed the documented-reason requirement and the persistent high risk tier after reinstatement	Retained; reason surfaced in the audit export
Usability score	Mean SUS score above the 70-point target across participants	Meets NFR-07

Table 25: UAT Feedback Summary

7.9 Requirements Validation Matrix

Requirement	Validating Evidence	Status
FR-01 Digital submission with tariff auto-population	IT-01/IT-02; submission screen; UT-06 tariff resolution	Validated
FR-02 Automated eligibility validation	UT-04; IT-01 pre-check stage	Validated
FR-03 Provider accreditation verification	UT-05; IT-06 post-suspension submission blocked	Validated
FR-04 XGBoost scoring with persisted score and version	IT-01/IT-02; Table 26 evaluation	Validated
FR-05 SHAP top-five plain-language display	IT-03; section 7.7.3 consistency review	Validated
FR-06 Configurable threshold flagging and routing	IT-02; section 7.7.2 threshold analysis	Validated
FR-07 Officer decisions with mandatory comments	UT-13; IT-04/IT-05	Validated
FR-08 Administrator CRUD under RBAC	UT-12; system security testing 7.6.3	Validated
FR-09 Append-only audit logging	IT-03/IT-05; privilege probe in 7.6.3	Validated
FR-10 PDF and CSV reporting	IT-08; UT-14	Validated
FR-11 Real-time claim status tracking	IT-10	Validated

FR-12 Automatic suspension and audited reinstatement	UT-07–UT-10; IT-06/IT-07	Validated
--	--------------------------	-----------

Table 26: Requirements Validation Matrix

7.10 Chapter Summary

Testing verified the implemented system at every level. Fourteen unit tests confirmed the correctness of feature computations, pre-check rules, suspension logic, authentication, and validation; ten integration scenarios confirmed the end-to-end claim lifecycle including the suspension trigger, audited reinstatement, and degraded-mode start-up; system testing confirmed the performance bound of NFR-03 and the security guarantees of NFR-01 and NFR-02, including the append-only audit log at the privilege level. The tuned XGBoost model outperformed the logistic regression baseline on every metric on the held-out test set, the 0.70 threshold was retained after explicit trade-off analysis, and per-claim explanations were consistent with the injected anomaly patterns. UAT met the usability target with a mean SUS above 70, and the requirements validation matrix records passing evidence for all twelve functional requirements. The verified system provides the basis for the conclusions and recommendations of Chapter Eight.

CHAPTER EIGHT: CONCLUSION AND RECOMMENDATIONS

8.1 Introduction

This final chapter draws the project to a close. It summarises what was built and found, assesses the achievement of each specific objective against documented evidence, states the contributions of the study, acknowledges its limitations honestly, and sets out recommendations for the Social Health Authority together with directions for future work.

8.2 Summary of the Project

This project set out to design, implement, and evaluate a machine learning pipeline for automated health insurance claims verification within Kenya's Social Health Authority, in response to the documented weaknesses of the current process: slow manual verification, undetected anomalous claims, unexplained automated signals that officers cannot trust, and the absence of provider-level enforcement. The delivered system is a functional prototype in which every submitted claim passes deterministic eligibility, provider, and tariff pre-checks; is scored by a single, grid-search-tuned XGBoost model over a 17-feature engineered vector trained with SMOTE-balanced data; and is explained through per-claim SHAP attributions rendered in plain language. Flagged claims are routed to a human review queue where decisions require mandatory comments and every action including views; is written to an append-only audit log. An automated provider monitoring module converts sustained claim-level flags into proportionate institutional action, suspending any provider that accumulates ten flags in a rolling thirty-day window, under a dedicated system identity, with reinstatement reserved to administrators and requiring a documented reason. The system was implemented on a four-tier architecture web interface, FastAPI application with JWT authentication and role-based access control across four roles, Python machine learning pipeline, and PostgreSQL database and verified through unit, integration, system, and user acceptance testing, with the model evaluated against a baseline on a held-out test set.

8.3 Achievement of Objectives

Each specific objective from Chapter One is assessed against the documented evidence:

Specific Objective	Evidence of Achievement	Status
Analyse current SHA verification processes and identify gaps and requirements	Chapter Four: stakeholder findings, current-system analysis, feasibility study, and requirements FR-01 to FR-12 and NFR-01 to NFR-07	Achieved
Design a pre-processing and 17-feature engineering pipeline for structured claims data	Chapter Three section 3.6 and Chapter Five section 5.4; implemented in Chapter Six 6.7.1 and unit-tested (UT-01 to UT-03)	Achieved
Implement and train an XGBoost model with SMOTE for class imbalance	Chapter Six 6.7.2: stratified split, SMOTE on training data only, grid search under 5-fold cross-validation, MLflow tracking	Achieved
Integrate SHAP explainability for every flagged claim	Chapter Six 6.7.3 and Chapter Seven 7.7.3: stored SHAP vectors, plain-language top-five rendering, consistency review	Achieved
Develop a functional end-to-end prototype with RBAC, CRUD, reporting, and automated provider suspension	Chapters Five and Six: six routers, four roles, live frontend, reporting service, suspension module; validated by IT-01 to IT-10	Achieved
Evaluate model performance on a held-out test set	Chapter Seven Table 26: XGBoost exceeding the baseline on accuracy, precision, recall, F1-score, and AUC-ROC; threshold analysis in 7.7.2	Achieved
Document and address ethical considerations	Chapters Two and Three: consent procedures, anonymised and synthetic data, encryption and RBAC, bias assessment plan, human-in-the-loop safeguards	Achieved

Table 27: Achievement of Specific Objectives

8.4 Contributions of the Study

- **Applied contribution:** a documented, working reference implementation of explainable machine learning claims verification on a realistic SHA data schema from claim submission through scoring, explanation, human review, and audit which SHA or subsequent researchers can extend rather than rebuild.
- **Methodological contribution:** evidence for the single-model, deep-optimisation strategy over comparative shallow evaluation in a time-boxed project: committing to XGBoost released the capacity that funded rigorous feature engineering, threshold calibration, explainability integration, and complete system delivery.

- **Design contribution:** the automated provider suspension mechanism with auditable human oversight a rolling-window threshold, a dedicated system actor identity in an append-only log, and administrator-controlled, reason-documented reinstatement for which no equivalent was found in the reviewed literature or systems.
- **Practical human-factors finding:** stakeholder testing confirmed that plain-language, per-claim explanation is the decisive adoption factor for officer trust, supporting the design principle that the model recommends and explains while the human decides.

8.5 Limitations of the Study

- Model training and evaluation relied on synthetic data generated to reflect SHA's schema, supplemented by public insurance datasets; reported performance is indicative, and retraining and re-evaluation on operational SHA data is a precondition for any deployment.
- The prototype was tested at development scale; production concerns load balancing, database replication and backup, monitoring, and formal availability engineering were out of scope.
- The suspension threshold of ten flags in thirty days was set conservatively from stakeholder consultation rather than derived from operational loss data, and should be recalibrated against real base rates.
- Bias assessment was limited to the demographic and geographic variables present in the synthetic data; a complete fairness audit requires operational data with richer covariates.
- The purposive stakeholder sample informs requirements and acceptance but does not support statistical generalisation across SHA nationally.

8.6 Recommendations

On the basis of the findings, the following recommendations are made to the Social Health Authority:

- SHA should pilot the pipeline on historical claims data under a formal data-sharing agreement, retraining the model and recalibrating the flagging threshold against actual anomaly base rates before any operational use.
- The append-only audit log and the mandatory-comment discipline should be adopted as institutional policy independent of the machine learning component; they materially strengthen accountability at negligible cost.

- The suspension threshold and rolling window should be governed by a published policy, reviewed periodically against false-positive rates, and accompanied by a documented provider appeal procedure.
- Officer decisions accumulated through the review workflow should be curated as labelled training data, creating a feedback loop for periodic model retraining under version control with MLflow.
- Deployment should be accompanied by officer training centred on interpreting the SHAP explanations, and by publication of a model card documenting the model's intended use, performance, and known limitations.
- A Data Privacy Impact Assessment should be completed and the fairness audit extended before deployment, in line with the Data Protection Act (2019) obligations identified in Chapters Two and Three.

8.7 Future Work

- Unsupervised anomaly detection alongside the supervised model, to surface novel patterns absent from labelled history.
- Graph-based analysis of provider-patient-facility relationships to detect collusive patterns invisible to per-claim features.
- Concept-drift monitoring with automated alerts when live feature distributions diverge from the training distribution, feeding the quarterly retraining cycle of NFR-06.
- A provider-facing appeal and evidence-upload workflow integrated with the reinstatement process.
- Integration with national registries facility master lists and member registers through official interfaces once available, replacing seeded reference data.
- A formal fairness study on operational data, including counterfactual testing across counties and facility tiers.

8.8 Conclusion

The project demonstrates that an explainable, human-supervised machine learning pipeline for claims verification is technically feasible, operationally acceptable to its intended users, and implementable within the resources of a final-year project when scope is disciplined. The combination that proved decisive was not any single component but their integration:

deterministic pre-checks reserving the model for genuine judgement; a single deeply optimised XGBoost classifier; SHAP explanations that convert scores into reasons officers can act on; an append-only audit trail that makes every action accountable; and a conservative, reversible, fully audited provider suspension mechanism that closes the loop from detection to enforcement. Applied to operational data under the recommendations above, this pipeline offers the Social Health Authority a practical path towards faster verification, reduced exposure to anomalous claims, and greater trust in the integrity of Kenya's universal health coverage system.

REFERENCES

- Bauder, R. A., Khoshgoftaar, T. M., & Seliya, N. (2017). A survey on the state of healthcare upcoding fraud analysis and detection. *Health Services and Outcomes Research Methodology*, 17(1), 31–55.
- Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794.
- Doetinchem, O., Carrin, G., & Evans, D. (2010). Thinking of introducing social health insurance? Ten questions (World Health Report Background Paper No. 26). World Health Organization.
- Joudaki, H., Rashidian, A., Minaei-Bidgoli, B., Mahmoodi, M., Geraili, B., Nasiri, M., & Arab, M. (2015). Using data mining to detect health care fraud and abuse: A review of literature. *Global Journal of Health Science*, 7(1), 194–202.
- Lundberg, S. M., & Lee, S. I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems*, 30, 4765–4774.
- Ministry of Health, Kenya. (2020). Kenya national eHealth policy 2016–2030. Government of Kenya.
- Ministry of Health, Kenya. (2023). The Social Health Insurance (General) Regulations: The Social Health Insurance Act (No. 16 of 2023). Government of Kenya.
- Ngai, E. W. T., Hu, Y., Wong, Y. H., Chen, Y., & Sun, X. (2011). The application of data mining techniques in financial fraud detection: A classification framework and an academic review of literature. *Decision Support Systems*, 50(3), 559–569.
- Raghupathi, W., & Raghupathi, V. (2014). Big data analytics in healthcare: Promise and potential. *Health Information Science and Systems*, 2(1), 3.
- Republic of Kenya. (2019). The Data Protection Act, 2019 (No. 24 of 2019). Government of Kenya.
- Social Health Authority. (2024). Benefits and services. <https://www.sha.or.ke/benefits/>

Witten, I. H., Frank, E., Hall, M. A., & Pal, C. J. (2016). Data mining: Practical machine learning tools and techniques (4th ed.). Morgan Kaufmann.

World Health Organization. (2024). Health financing resources. <https://www.who.int/health-topics/health-financing>

Appendix A: Stakeholder Questionnaire

Section A: Respondent Information

1. What is your primary role in relation to SHA health insurance claims?

- SHA Claims Verification Officer
- Healthcare Provider Administrator
- Healthcare Financial Officer
- Health Information System Administrator

2. Years of experience working with health insurance claims systems: _____

3. Frequency of interaction with SHA claims systems:

- Daily Weekly Monthly Occasionally

Section B: Current Verification System Challenges

4. Rate the efficiency of the current SHA claims verification process (1 = Very inefficient, 5 = Very efficient): _____

5. Typical time from claim submission to payment receipt:

- Less than 14 days 14–30 days 31–90 days More than 90 days

6. Verification failures encountered (select all that apply):

- Billing amounts not checked against SHA tariff schedules
- Diagnosis-treatment mismatches not detected at submission
- Ghost patient / ineligible beneficiary claims processed
- Duplicate claims approved for payment
- Newly registered provider anomalies not flagged
- No real-time eligibility confirmation available

7. Estimated percentage of processed claims containing errors or fraud: _____%

Section C: ML-Based Verification System Expectations

8. Would an automated ML-based claims verification system improve SHA's processing efficiency? Yes / No / Unsure

9. Rank the following system features by importance (1 = most important):

- Automated eligibility and coverage verification ____
- Real-time billing compliance checking against SHA tariffs ____
- Anomaly flagging with explanation of why a claim was flagged ____
- Audit trail for all verification decisions ____
- Management dashboard with processing analytics ____

10. Concerns about an ML-driven claims verification system (open response):

Appendix B: Interview Guide

Target: SHA Administrators, Claims Verification Officers, Financial Managers

Format: Semi-structured, 30 - 45 minutes, audio-recorded with participant consent

1. Walk me through the complete claims verification workflow from submission receipt to approval decision. Where are the main bottlenecks?
2. What specific types of verification failures: billing errors, eligibility issues, clinical inconsistencies are most common and most difficult to detect manually?
3. How does the current system handle claims from newly registered providers or unusual claim volumes from established providers?
4. What data fields are consistently available in submitted claims, and which are frequently missing or submitted in inconsistent formats?
5. If the system automatically scored each claim and highlighted the reasons for concern, how would that change the verification workflow for your officers?
6. What would need to be true about an automated verification system for SHA officers to trust and act on its outputs?
7. What reporting and analytics capabilities would be most valuable to SHA management from a claims verification system?

Appendix C: System Requirements Specification

Functional Requirements

Ref	Requirement	Acceptance Criteria
FR-01	Claims Submission: Healthcare providers submit structured claims digitally via the web portal with all mandatory fields.	Claims with all required fields stored in database within 5 seconds of submission.
FR-02	Eligibility Check: System automatically verifies patient SHA membership status and service eligibility at submission.	Eligibility result returned within 3 seconds; ineligible claims rejected with specific reason code.
FR-03	XGBoost Verification Scoring: Every validated claim is scored by the XGBoost model returning a verification flag probability (0–1).	Score and SHAP values returned within 3 seconds and logged in the Verification table.
FR-04	Automated Flagging: Claims with scores above configurable threshold (default 0.70) are routed to officer review queue.	Flagged claims visible in officer dashboard within 10 seconds; notification triggered.
FR-05	SHAP Explanation Display: Officer review interface shows top 5 SHAP features driving each flagged claim's score.	SHAP waterfall chart rendered for every flagged claim in officer review screen.
FR-06	Officer Review and Decision: Officers view full claim, SHAP explanation, add comments, and approve or escalate.	Decision logged in AuditLog with officer ID, timestamp, action, and comments.
FR-07	CRUD Operations: Administrators manage provider records, user accounts, and SHA tariff schedules.	All CRUD operations complete with validation; unauthorised role access blocked.
FR-08	Audit Log: Every system action logged with actor ID, timestamp, action type, and data state.	100% of system events recorded; log is tamper-evident and exportable.
FR-09	Reporting Dashboard: Real-time statistics on claims volumes, verification rates, processing times, and provider performance.	Dashboard loads within 5 seconds; data refresh within 60 seconds.

FR-10	Claim Status Tracking: Providers track real-time status of all submitted claims through the provider portal.	All status transitions (submitted, verified, flagged, approved, rejected, paid) displayed in real time.
-------	--	---

Table 28: Functional Requirements

Non-Functional Requirements

NFR-01 Security	TLS 1.2+ for all data in transit; bcrypt password hashing; sensitive fields encrypted at rest.	Zero plaintext transmission. Account lockout after 5 consecutive failed login attempts.
NFR-02 Access Control	RBAC enforced for all system functions across four roles: Provider, SHA Officer, Administrator, Finance Officer.	Unauthorised role access attempts blocked and logged. Role matrix documented.
NFR-03 Performance	XGBoost verification score returned within 3 seconds for 95% of claims under normal load.	Load testing confirms <3 second response for up to 50 concurrent claim submissions.
NFR-04 Availability	99% uptime during operational hours (7am–8pm EAT, Monday–Saturday).	Monitoring logs confirm downtime not exceeding 7.2 hours per month.
NFR-05 Privacy	Compliance with Kenya's Data Protection Act (2019): data minimisation, purpose limitation, anonymisation in development.	Data Privacy Impact Assessment completed and documented before system deployment.
NFR-06 Maintainability	XGBoost model retrained quarterly using newly accumulated verified claims data.	MLflow experiment tracking confirms model versioning; retraining pipeline scheduled and documented.
NFR-07 Usability	SHA officer interface achieves minimum SUS (System Usability Scale) score of 70 in UAT.	UAT results from minimum 5 SHA officer participants confirm SUS score ≥ 70 .

Table 29: Non-Functional Requirements

Appendix D: Gantt Chart

The Gantt chart below consolidates the project activities into a four-month schedule, showing the month(s) during which each activity is undertaken.

Activity	Month 1 May	Month 2 June	Month 3 July	Month 4 August
Phase 1: Planning & Research				
Proposal finalisation & supervisor onboarding				
Ethics review submission				
Literature deep dive				
Stakeholder questionnaires & interviews (18)				
Current system observation & data schema mapping				
Phase 2: Data & ML Pipeline				
Dataset preparation & exploratory analysis				
Data preprocessing pipeline (SMOTE, encoding, scaling)				
Feature engineering (amount_ratio, provider anomaly, clinical match)				
XGBoost model implementation & initial training				
Hyperparameter tuning via grid search & cross-validation				
Model evaluation: F1-score, AUC-ROC, SHAP analysis				
Phase 3: System Development				
Database design & backend API (FastAPI)				
Claims submission & XGBoost verification modules				
Officer dashboard & SHAP explanation interface				
CRUD operations & reporting module				
Phase 4: Testing & Evaluation				
Integration testing & UAT (SUS evaluation)				
Phase 5: Writing & Submission				
Final report writing: Chapters 4-8				
Submission, presentation & defence rehearsal				

Table 30: Gantt Chart

Appendix E: User Manual

D.1 System Requirements and Installation

The system requires Python 3.11 or later, PostgreSQL 15, and a modern web browser. Installation proceeds as follows: create the database and run `database/schema.sql` followed by `database/permissions.sql`; create a Python virtual environment in the backend directory and install dependencies with `pip install -r requirements.txt`; copy `.env.example` to `.env` and supply the database connection string and JWT secret; run `python seed_users.py` once to provision the demonstration accounts; start the server with `uvicorn app.main:app --reload`; and open `frontend/index.html` in a browser. Interactive API documentation is available at `/docs`.

D.2 Signing In

Enter the email address and password for your role and select Sign in. The interface reveals only the tabs permitted to your role. Demonstration accounts exist for the officer, administrator, finance, and provider roles; passwords are set by the seed script and should be changed for any non-demonstration use.

D.3 Submitting a Claim (Provider)

On the Submit Claim tab, enter the member's SHA number and your provider identifier, select the service date, and choose the diagnosis and procedure codes; the applicable SHA tariff is fetched and displayed automatically. Enter the claimed amount, a warning appears immediately if it exceeds twice the tariff. Select Submit claim. A result banner reports the outcome: verified claims proceed towards approval, while flagged claims are routed to officer review. Claim status can be tracked at any time from the status view using the claim identifier.

D.4 Reviewing Flagged Claims (Officer)

The Officer Queue tab shows today's metrics and all flagged claims ordered by score. Select Review on any claim to open its detail view, which presents the claim data, the pre-check results, and the explanation panel listing the top five factors behind the score in plain language. Enter your comments; these are mandatory, and choose Approve, Reject, or Escalate. Every view and decision are recorded in the audit log.

D.5 Administration (Administrator)

The Admin tab provides provider management with search and filters: add new providers, suspend or delete existing ones, and monitor each provider's flag accumulation and risk tier. A provider suspended automatically by the system can be reinstated only from this screen, and a documented reason of at least ten characters is required; the provider's risk tier remains elevated after reinstatement. The flagging threshold used by the pipeline is configurable from the administration settings.

D.6 Reports

The Reports tab generates the verification summary PDF and the audit trail and flagged claims CSV exports over a selectable date range, and displays the at-risk providers widget showing progress toward the suspension threshold. Files download directly through the browser.

D.7 Troubleshooting

- If sign-in fails with valid credentials, confirm the backend is running and the database is reachable.
- If tariff lookup returns no result, the diagnosis-procedure pair is not in the tariff schedule for the facility tier; verify the codes.
- If submission is rejected with a provider error, the provider may be suspended; contact an administrator.
- If reports fail to download, confirm the selected date range contains data.